Simula Training Data Framework
Implementation Specification

Synthetic Data Generation for Text-to-SQL

Ref: `SIM-TRAIN-2026-v1.2`

SAP-OSS AI Training Team

April 2026

Abstract

This specification defines the full implementation of the **Simula** reasoning-driven synthetic training data generation framework for SAP HANA Cloud. The system addresses a critical gap identified in the existing training pipeline: *there is no direct pipe to HANA to generate training datasets for model training.* Three orthogonal components are specified in full:

- a **HANA Schema Extraction Pipeline** that connects to SAP HANA Cloud via the AI Core PAL MCP service, queries `SYS.TABLE_COLUMNS` for live metadata, and populates a schema registry;
- a **Taxonomy Generation Engine** that implements Algorithm 1 from the Simula research paper—factor identification, breadth-first expansion with Best-of-N proposals, and generator-critic refinement;
- a **Training Data Generation Engine** that implements Algorithm 2—taxonomic mix sampling, meta-prompt generation, complexification, and double-critic filtering.

All three components consume a single vLLM TurboQuant backend via the OpenAI-compatible API and output JSONL training data suitable for LoRA fine-tuning. The specification targets three application domains: Arabic document processing (invoice extraction), Trial Balance review (variance analysis and LLM commentaries), and ESG analytics (financed emissions classification).

Reading order. Chapter 1 situates the work; Chapter 2 defines the data structures; Chapter 3 covers the HANA extraction pipeline; Chapter 4 covers the taxonomy engine; Chapter 5 covers the data generator; Chapter 6 specifies environment and CLI; Chapter 7 covers the evaluation framework; Chapter 8 provides references and traceability.

Conventions. File paths are typeset in `monospace` and are relative to the repository root `/Users/user/Documents/sap-oss`. Enum values in running text appear in SMALL CAPS. Python code listings include line numbers for cross-reference.

Contents

Executive Summary . . . 7

1 Overview and Scope **8**
1.1 The Problem . . . 8
1.2 Goals . . . 8
1.3 In-Scope Domains . . . 9
1.4 System Context . . . 9
1.5 Research Paper Traceability . . . 9
1.6 Out of Scope . . . 10

2 Canonical Data Schema **11**
2.1 Layout of `src/training/pipeline/` . . . 11
2.2 Training Example Record . . . 11
2.3 Schema Registry . . . 11
2.4 Taxonomy Structure . . . 12
2.5 Enumerations . . . 12
2.6 Unified Schema Registry . . . 13

3 HANA Schema Extraction Pipeline **14**
3.1 Pipeline at a Glance . . . 14
3.2 Stage 1 — MCP Discovery . . . 14
3.3 Stage 2 — Table Enumeration . . . 14
3.4 Stage 3 — Column Extraction . . . 15
3.5 Stage 4 — Registry Population . . . 15
3.6 Stage 5 — JSON Export . . . 16

4 Taxonomy Generation Engine **17**
4.1 Algorithm 1: Reasoning-Driven Taxonomy Generation . . . 17
4.2 Step 1: Factor Identification . . . 17
4.3 Step 2: Breadth-First Expansion . . . 18
4.4 Step 3: Best-of-N Proposal . . . 18
4.5 Step 4: Critic Refinement . . . 19
4.6 Step 5: Level Planning . . . 19
4.7 Worked Example: Trial Balance Taxonomy . . . 19

5 Training Data Generation Engine **20**
5.1 Algorithm 2: Agentic Data Synthesis . . . 20
5.2 Step 1: Mix Sampling (Global Diversity) . . . 20
5.3 Step 2: Meta-Prompt Generation (Local Diversity) . . . 21
5.4 Step 3: Complexification . . . 21
5.5 Step 4: Example Generation . . . 21
5.6 Step 5: Double-Critic Filtering . . . 22
5.7 Output Format . . . 23

6 Environment and CLI **24**
6.1 Configuration Table . . . 24
6.2 CLI Usage . . . 24
6.3 Output Files . . . 25

7 Evaluation Framework **26**
7.1 Calibrated Complexity Scoring (Algorithm 3) . . . 26
7.1.1 Elo Rating Algorithm . . . 26
7.1.2 Usage . . . 27
7.2 Taxonomic Coverage Evaluation . . . 27
7.3 Embedding-Based Diversity Metrics . . . 28
7.4 Critic Calibration Validation . . . 28
7.5 Student-Teacher Gap Analysis . . . 29
7.6 Adaptive Features (v1.2) . . . 29
7.6.1 Adaptive Complexity Ratio (Figure 7) . . . 29
7.6.2 Adaptive Critic Thresholding (Figure 3b) . . . 30
7.6.3 Automatic Coverage Gap Filling . . . 30
7.6.4 Diversity Optimizer . . . 30
7.6.5 Taxonomy Quality Gate (Section B.1, Table 2) . . . 31
7.7 Evaluation Environment Variables . . . 32
Worked Example . . . 33

8 References **37**
8.1 Source Documents . . . 37
8.2 Implementation Files . . . 37
8.3 Tools . . . 38

9 Test Fixture Specification **39**
9.1 Overview . . . 39
9.2 Fixture organisation . . . 39
9.3 Arabic AP fixtures . . . 40
9.3.1 Invoice fixtures . . . 40
9.3.2 OCR fixtures . . . 40
9.4 Trial Balance fixtures . . . 41
9.4.1 TB extract fixtures . . . 41
9.4.2 Variance fixtures . . . 41
9.5 Simula fixtures . . . 41
9.5.1 Taxonomy fixtures . . . 41
9.5.2 Training example fixtures . . . 42
9.6 Common fixtures . . . 42
9.6.1 Mock LLM responses . . . 42
9.6.2 Schema copies . . . 42
9.7 Data sanitisation requirements . . . 42
9.8 Fixture validation . . . 43
9.8.1 Schema validation . . . 43
9.8.2 Coverage verification . . . 43
9.9 Implementation checklist . . . 43

10 Testing Specification **44**
10.1 Test coverage requirements . . . 44
10.2 Test fixtures . . . 44
10.2.1 Taxonomy fixtures . . . 44
10.2.2 Training example fixtures . . . 45
10.3 Unit test cases . . . 45
10.3.1 Taxonomy traversal . . . 45
10.3.2 Complexity scoring . . . 45
10.3.3 Critic evaluation . . . 45
10.3.4 Diversity metrics . . . 45
10.3.5 Adaptive features . . . 46
10.3.6 HANA type handling . . . 46
10.4 Integration test suite . . . 47
10.4.1 End-to-end scenarios . . . 47
10.5 Performance benchmarks . . . 47
10.5.1 Reference hardware profile . . . 48
10.6 Quality metrics tests . . . 48
10.7 CI/CD integration . . . 48
10.8 Acceptance criteria . . . 48

11 Project Roadmap **50**
Scope of this chapter . . . 50
11.1 MoSCoW classification . . . 50
11.2 Dependency graph . . . 50
11.3 Project milestones . . . 50
11.4 Launch tiers and exit gates . . . 50
11.5 Kill-switch conditions . . . 54
11.6 Shared-platform dependencies . . . 54
11.7 v1.3 expansion trigger . . . 54
11.8 Governance and business-case cadence . . . 54

12 Agent Implementation Instructions **57**
12.1 Purpose and audience . . . 57
12.2 Rules of Engagement for AI Agents . . . 57
12.3 Platform Overview — One Codebase, One Instance . . . 57
12.3.1 Integration Stance . . . 57
12.3.2 Platform Integration Map . . . 58
12.4 Transactional Sequence Diagrams . . . 58
12.4.1 Universal Governance Handshake . . . 58
12.4.2 Arabic AP Golden Path . . . 58
12.5 Compute and Resource Alignment . . . 58
12.6 Human-in-the-Loop (HITL) Critical Gates . . . 59
12.7 Registry Integrity & Live Status . . . 59
12.8 Master Implementation Manifest (Union) . . . 59
12.9 Governance Obligations (MAS MGF) . . . 60
12.10 FMEA: Failure Mode and Effects Analysis for AI . . . 60
12.11 This domain in context . . . 61
12.12 Schema contribution . . . 61
12.13 Persona and MCP allow-list . . . 61
12.14 Implementation tasks for this domain . . . 61
12.15 Governance obligations applied to this domain . . . 63
12.16 Cross-spec integration — full detail . . . 64

12.16.1 With Arabic AP 64
12.16.2 With Trial Balance 64
12.16.3 With Regulations 64
12.16.4 With Simula 64
12.17 Wave assignment and sequencing 64
12.18 Acceptance criteria (platform-wide) 65
12.19 References 65

13 Global Wave-Dependency Manifest **66**
13.1 Critical Path Visualization (Waves 1–3) 66
13.2 Inter-Domain Prerequisite Matrix 66

List of Tables

1.1 Current vs. Target Training Data Pipeline . . . 8
1.2 Application Domains . . . 9
1.3 Traceability: Research Paper to Implementation . . . 10

2.1 Pipeline Module Layout . . . 11
2.2 Key Enumerations . . . 12

4.1 Sample Taxonomy for Trial Balance Domain . . . 19

6.1 Environment Variables . . . 24
6.2 Pipeline Output Files . . . 25

7.1 Evaluation Framework Environment Variables . . . 32
7.2 Generated Taxonomy for "query_type" Factor . . . 34
7.3 Evaluation results for Trial Balance dataset (reference run `simula-v1.2-worked-example`, 2025-11-15). . . . 35

8.1 Source Documents . . . 37
8.2 Implementation Files . . . 37
8.3 Tools and Dependencies . . . 38

9.1 Required invoice fixture categories. . . . 40
9.2 OCR fixture confidence levels. . . . 40
9.3 Required TB extract fixture categories. . . . 41
9.4 Training example fixture requirements. . . . 42
9.5 Data sanitisation rules. . . . 42

10.1 Simula test coverage targets. . . . 44
10.2 Training example fixture requirements. . . . 45
10.3 HANA spatial type → scalar method mapping. . . . 46
10.4 Simula performance targets on reference hardware. . . . 47

11.1 MoSCoW prioritisation (shared across the four SAP OSS Finance projects). . . . 51
11.2 Simula v1.2 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't. . . . 52
11.3 Simula launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier. . . . 53
11.4 Simula kill-switch. . . . 55
11.5 Shared-platform dependencies. . . . 56

12.1 Infrastructure Reference Profiles. . . . 59
12.2 FMEA for Simula AI Pipeline. . . . 60

13.1 Normative Cross-Domain Prerequisites. . . . 67

List of Figures

1.1 System context. HANA Cloud (green) provides live schema metadata via the MCP protocol (blue). The vLLM backend (purple) powers taxonomy expansion and data generation. Output is JSONL training data (gold). 9

3.1 The five extraction stages; each emits a well-typed payload to the next. 14

4.1 Taxonomy generation stages with iterative refinement. 17

5.1 Data generation stages implementing Algorithm 2. 20

11.1 Simula dependency graph. Amber nodes are Must (critical path); blue are Should (consumer-corpus allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the v1.3 trigger in Section 11.7 fires). 52

12.1 Cross-domain interaction and unified data flow map. Detailed governance logic is defined in **??**. 58

12.2 Standard platform handshake for governed tool execution. Audit schemas are detailed in **??**. 58

12.3 Arabic AP sequence: Scanned PDF to Release. VAT rules are specified in **??**. . . 59

13.1 Cross-specification dependency graph: The blueprint for platform rollout. 66

Executive Summary

Business Problem

The SAP OSS Studio AI platform requires high-quality training data to fine-tune Text-to-SQL models for domain-specific applications (Trial Balance, Arabic invoices, ESG analytics). The current pipeline uses **static Excel files** as schema sources, creating three critical issues:

1. **Schema drift:** Training data becomes stale when HANA schemas evolve.
2. **Coverage gaps:** Manual templates miss edge cases and rare query patterns.
3. **Quality variance:** No systematic quality assurance for generated examples.

Solution: Simula Framework

This specification implements the **Simula** framework from Davidson et al. (2026), a research-validated methodology for reasoning-driven synthetic data generation. The framework addresses all three issues through:

- **HANA-Direct Extraction** Live schema queries via AI Core PAL MCP service.
- **Taxonomic Coverage** Hierarchical factor trees ensure systematic diversity.
- **Agentic Quality Control** Double-critic filtering with Elo-calibrated complexity.

Key Metrics and Targets

Metric	Paper Target	Implementation	Source
Global diversity	0.48–0.54 cosine	Target: 0.52	Figure 4
Local diversity	0.10–0.30 cosine	Target: 0.20	Figure 4
Complexity ratio	50% (c=0.5)	Adaptive 0.2–0.5	Figure 7
Critic acceptance	$p(y) > 0.8$	Threshold: 0.5–0.7	Figure 3
Taxonomy completeness	74% (Simula)	Min: 70%	Table 2
Taxonomy soundness	97% (Simula)	Min: 90%	Table 2
Saturation ratio	83%	83%	Section 4.3

Expected Outcomes

1. **10× training data** with consistent quality (100k+ examples).
2. **Full taxonomic coverage** no systematic gaps in query patterns.
3. **Calibrated complexity** downstream model benefits from high-complexity data.
4. **Audit trail** every example traceable to taxonomy nodes and meta-prompts.

Chapter 1

Overview and Scope

1.1 The Problem

The SAP OSS training pipeline requires high-quality Text-to-SQL training data that reflects the actual schema structure of SAP HANA Cloud tables. The current implementation in `src/training/pipeline/` uses metadata from **Excel files** (`DATA_DICTIONARY.xlsx`, `ESG_DATA_DICTIONARY.xl` that were originally exported from HANA or defined manually.

While the `HanaClient` in `src/training/pipeline/hana_client.py` has the capability to query `SYS.TABLE_COLUMNS` for live schema metadata, this HANA-direct extraction is **not currently integrated** into the main training data generation pipeline.

Table 1.1: Current vs. Target Training Data Pipeline

Dimension	**Current State**	**Target State**
Schema Source	Static Excel/CSV files	Live HANA Cloud metadata
Extraction Method	Manual export	MCP-based automated query
Training Data	Template-based expansion	Reasoning-driven synthesis
Diversity Control	None	Taxonomic coverage
Quality Assurance	None	Double-critic filtering
Complexity Control	None	Calibrated Elo scoring

1.2 Goals

The specification targets three outcomes:

1. **HANA-Direct Extraction.** Every training dataset is generated from live schema metadata queried via the AI Core PAL MCP service.
2. **Taxonomic Coverage.** The conceptual space of the target domain is mapped using hierarchical taxonomies, ensuring global diversity.
3. **Reasoning-Driven Synthesis.** Each training example is generated from first principles using agentic refinement, ensuring quality, local diversity, and controlled complexity.

1.3 In-Scope Domains

Table 1.2: Application Domains

Domain	HANA Schema	Training Task
Arabic Document Processing	`AP_INVOICES`, `VAT_CHECKLISTS`	Invoice field extraction, VAT compliance
Trial Balance Review	`TB_VARIANCES`, `GL_BALANCES`	Variance analysis, commentary generation
ESG Analytics	`ESG_METRICS`, `FINANCED_EMISSIONS`	Scope classification, net-zero alignment

1.4 System Context

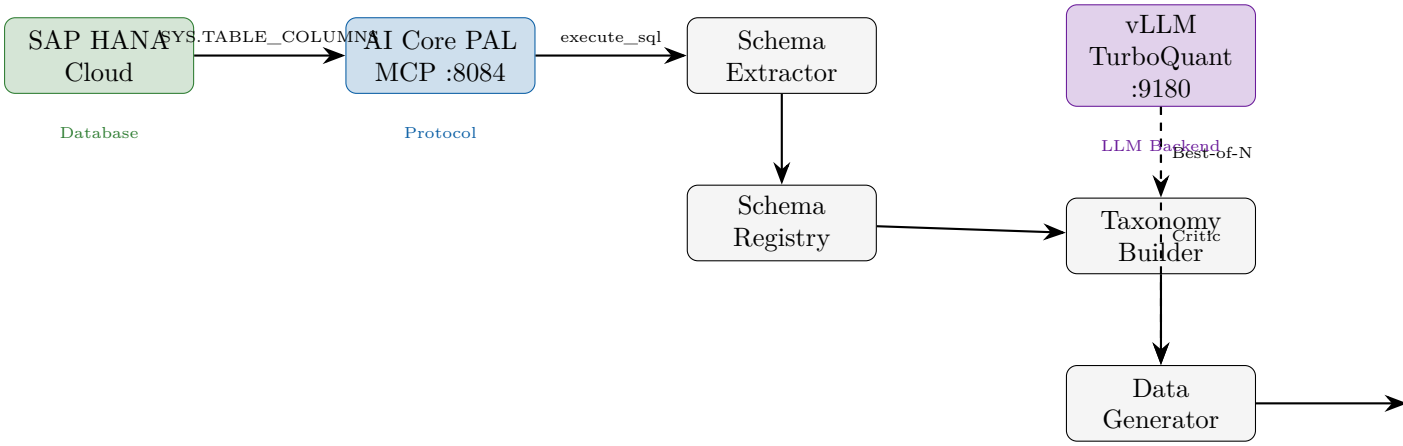

Figure 1.1: System context. HANA Cloud (green) provides live schema metadata via the MCP protocol (blue). The vLLM backend (purple) powers taxonomy expansion and data generation. Output is JSONL training data (gold).

1.5 Research Paper Traceability

This specification implements the Simula framework from Davidson et al. (2026), "Reasoning-Driven Synthetic Data Generation and Evaluation," published in Transactions on Machine Learning Research (TMLR). The following table maps paper sections to specification sections:

Table 1.3: Traceability: Research Paper to Implementation

Paper Reference	**Paper Finding**	**Spec Implementation**
Section 2.1, Figure 1	Taxonomy-based coverage	ğ4, Algorithm 1
Section 2.2, Algorithm 2	Agentic data synthesis	ğ5, Algorithm 2
Section 2.3, Algorithm 3	Calibrated complexity scoring	ğ7.1, Elo algorithm
Section 3.1, Figure 3	Double-critic validation	ğ5.5, ğ7.4
Section 3.3, Figure 4	Additive diversity	ğ7.3, Diversity analyzer
Section 4.3	83% saturation ratio	ğ7.5, Gap analysis
Figure 7	Complexity vs teacher strength	ğ7.6.1, Adaptive ratio
Table 2 (Appendix B)	Taxonomy quality metrics	ğ7.6.5, Quality gate
Appendix E.3	Elo algorithm details	ğ7.1.1, `_update_elo()`

1.6 Out of Scope

The following are **explicitly** out of scope for this document:

- Model fine-tuning and evaluation (covered by separate LoRA training specification).
- Downstream inference and routing (covered by `03-prompt-routing.tex`).
- Trial Balance business process (covered by `04-trial-balance-ai-specification.tex`).
- Arabic invoice pipeline (covered by the Arabic AP specification).

Chapter 2

Canonical Data Schema

2.1 Layout of `src/training/pipeline/`

Table 2.1: Pipeline Module Layout

Path	Purpose
`simula_config.py`	Configuration dataclasses for all components
`simula_llm_client.py`	OpenAI-compatible async LLM client
`hana_schema_extractor.py`	HANA Cloud schema extraction via MCP
`simula_taxonomy_builder.py`	Algorithm 1: Taxonomy generation
`simula_data_generator.py`	Algorithm 2: Training data synthesis
`schema_registry.py`	In-memory schema registry

2.2 Training Example Record

Each generated training example is a JSON object with the following structure:

```python
@dataclass
class TrainingExample:
    """A single training example for Text-to-SQL."""
    id: str                      # Unique identifier
    question: str                # Natural language question
    sql: str                     # Target SQL query
    schema_context: str          # Table/column descriptions
    taxonomy_path: list[str]     # Factor nodes used
    complexity_score: float      # 0.0 to 1.0
    meta_prompt: str             # The meta-prompt used
    is_complexified: bool        # Whether complexification applied
    critic_passed: bool          # Whether double-critic passed
    generation_metadata: dict    # Timestamps, model, etc.
```

Listing 2.1: TrainingExample dataclass (simula_data_generator.py)

2.3 Schema Registry

The `SchemaRegistry` holds table and column metadata extracted from HANA:

```python
@dataclass
class Column:
    name: str
```

```python
    data_type: str
    length: int | None
    nullable: bool
    description: str = ""

@dataclass
class TableSchema:
    schema_name: str
    table_name: str
    columns: list[Column]
    primary_key: list[str] = field(default_factory=list)
    description: str = ""
```

Listing 2.2: Schema registry structures (schema_registry.py)

2.4 Taxonomy Structure

Taxonomies are hierarchical trees where each node represents a factor of variation:

```python
@dataclass
class TaxonomyNode:
    id: str
    name: str
    description: str
    level: int
    parent_id: str | None
    children: list["TaxonomyNode"]
    examples: list[str]  # Sample values

@dataclass
class Factor:
    id: str
    name: str
    description: str
    taxonomy: "Taxonomy"

@dataclass
class Taxonomy:
    id: str
    factor: str
    root: TaxonomyNode
    depth: int
    total_nodes: int
```

Listing 2.3: Taxonomy structures (simula_taxonomy_builder.py)

2.5 Enumerations

Table 2.2: Key Enumerations

Enum	Values	Description
`LLMProvider`	VLLM, OPENAI, ANTHROPIC	LLM backend type
`SamplingStrategy`	UNIFORM, WEIGHTED, COVERAGE	How to sample taxonomy nodes

2.6 Unified Schema Registry

As of 18 April 2026, all schemas in this specification are published in the unified schema registry at `docs/schema/`. Migration status is *complete* (see `docs/schema/registry.json` field `migration.status`). This consolidation addresses cross-document schema divergence identified in the specification review.

Registry index. `docs/schema/registry.json` — Master index of all schemas across domains.

Common types. `docs/schema/common/base-types.schema.json` — Shared definitions (`Metadata`, `Currency`, etc.) referenced via JSON Schema `$ref`.

Consolidated enums. `docs/schema/common/enums.yaml` — Single source of truth for enumeration values used by all four specifications.

Simula schemas. `docs/schema/simula/` — Domain-specific schemas: `taxonomy.schema.json`, `training-example.schema.json`, `hana-schema-entry.schema.json`, `config.schema.json`, and `meta-prompt.schema.json`. All marked `status: stable`.

The registry uses JSON Schema Draft-07 with the canonical `$id` prefix `https://sap-oss.github.io/schema/`. Validation:

```
python3 docs/schema/validate.py --registry-check
python3 docs/schema/validate.py --domain simula \
    --schema training-example.schema.json --file example.json
```

See `docs/schema/README.md` for Python usage examples; the CI pipeline executes `-registry-check` on every pull request that touches `docs/schema/` or any spec chapter that references these schemas.

Chapter 3

HANA Schema Extraction Pipeline

3.1 Pipeline at a Glance

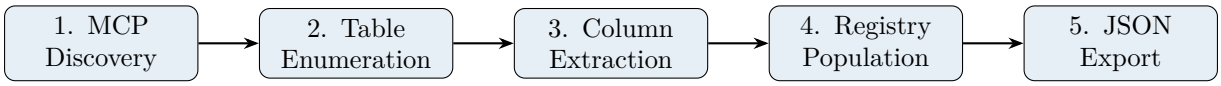

Figure 3.1: The five extraction stages; each emits a well-typed payload to the next.

3.2 Stage 1 — MCP Discovery

The extractor connects to the AI Core PAL MCP server and discovers available tools:

```
async def connect(self) -> None:
    """Establish MCP connection to AI Core PAL."""
    # Handle both HTTP->WS and HTTPS->WSS
    transport_url = self.config.mcp_url
    transport_url = transport_url.replace("https://", "wss://")
    transport_url = transport_url.replace("http://", "ws://")
    transport_url = f"{transport_url}/mcp/ws"

    self._transport = WebSocketClientTransport(transport_url)
    self._session = ClientSession(self._transport)

    await self._session.initialize()

    # Discover available tools
    tools_result = await self._session.list_tools()
    self._available_tools = {t.name for t in tools_result.tools}
```

Listing 3.1: MCP connection (hana_schema_extractor.py, lines 45–60)

Precondition. The MCP server must expose at least one of: `execute_sql`, `hana_tables`, or `list_tables`.

3.3 Stage 2 — Table Enumeration

Tables are discovered via the `hana_tables` MCP tool or by querying `SYS.TABLES`:

```
async def _discover_tables(self, schema: str) -> list[str]:
    """Discover tables in a schema."""
    if "hana_tables" in self._available_tools:
        result = await self._session.call_tool(
```

```
            "hana_tables",
            {"schema": schema}
        )
        return [t["table_name"] for t in result.content]
    else:
        # Fallback to direct SQL
        result = await self._execute_sql(
            'SELECT TABLE_NAME FROM SYS.TABLES '
            'WHERE SCHEMA_NAME = ?',
            [schema]
        )
        return [row["TABLE_NAME"] for row in result]
```

Listing 3.2: Table discovery (hana_schema_extractor.py, lines 85–105)

3.4 Stage 3 — Column Extraction

For each table, column metadata is extracted from `SYS.TABLE_COLUMNS`:

```
SELECT
    COLUMN_NAME,
    DATA_TYPE_NAME,
    LENGTH,
    IS_NULLABLE
FROM SYS.TABLE_COLUMNS
WHERE SCHEMA_NAME = ? AND TABLE_NAME = ?
ORDER BY POSITION
```

Listing 3.3: Column extraction SQL

3.5 Stage 4 — Registry Population

Extracted metadata is normalized and stored in the `SchemaRegistry`:

```
async def extract_table_schema(
    self, schema: str, table: str
) -> TableSchema:
    """Extract full schema for a single table."""
    columns_data = await self._get_table_columns(schema, table)

    columns = [
        Column(
            name=col["COLUMN_NAME"],
            data_type=col["DATA_TYPE_NAME"],
            length=col.get("LENGTH"),
            nullable=col.get("IS_NULLABLE", "TRUE") == "TRUE",
        )
        for col in columns_data
    ]

    return TableSchema(
        schema_name=schema,
        table_name=table,
        columns=columns,
    )
```

Listing 3.4: Registry population (hana_schema_extractor.py, lines 140–165)

3.6 Stage 5 — JSON Export

The registry exports to JSON for caching and debugging:

```
{
  "schemas": {
    "PAL_STORE": {
      "tables": {
        "ESG_METRICS": {
          "columns": [
            {"name": "METRIC_ID", "type": "INTEGER", ...},
            {"name": "SCOPE", "type": "NVARCHAR", ...}
          ]
        }
      }
    }
  },
  "extracted_at": "2026-04-18T08:00:00Z"
}
```

Listing 3.5: JSON export format

Chapter 4

Taxonomy Generation Engine

4.1 Algorithm 1: Reasoning-Driven Taxonomy Generation

The taxonomy builder implements Algorithm 1 from the Simula research paper. Given a dataset description y and target depth D, it produces a set of taxonomies $\mathcal{T} = \{T_i\}_{i=0}^{K}$ that map the conceptual space.

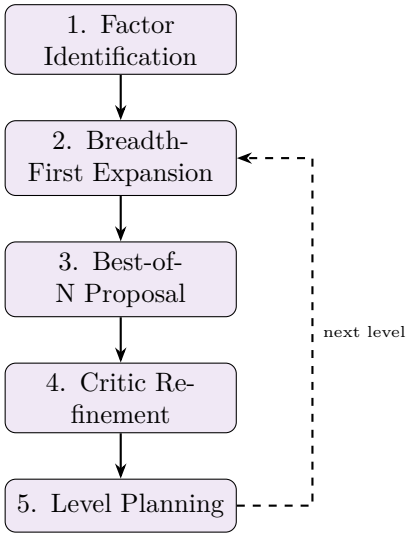

Figure 4.1: Taxonomy generation stages with iterative refinement.

4.2 Step 1: Factor Identification

Given schema metadata, the LLM proposes factors of variation:

```
SYSTEM: You identify the prime factors of variation for a
        Text-to-SQL training dataset.

USER:   Schema context:
        {schema_context}

        Identify 3-7 factors that capture the main dimensions
        of queries users might ask. Examples:
        - "query_complexity" (simple, join, aggregation, subquery)
        - "time_range" (point-in-time, period, YTD, MTD)
        - "entity_type" (customer, order, product, supplier)
```

```
        Output JSON: {"factors": [{"name": "...", "description": "..."
    }]}
```

Listing 4.1: Factor identification prompt

4.3 Step 2: Breadth-First Expansion

Each factor is expanded into a taxonomy tree. For each node, the LLM proposes children:

```
async def _expand_node(
    self, node: TaxonomyNode, context: str
) -> list[TaxonomyNode]:
    """Expand a single node with Best-of-N proposals."""

    # Step 1: Generate N proposals
    proposals = await self.llm.best_of_n(
        prompt=self._build_expansion_prompt(node, context),
        n=self.config.taxonomy.best_of_n,
        temperature=0.8,
    )

    # Merge proposals into candidate set
    candidates = self._merge_proposals(proposals)

    # Step 2: Critic refinement
    refined = await self._critic_refine(node, candidates, context)

    return refined
```

Listing 4.2: Node expansion (simula_taxonomy_builder.py, lines 180–220)

4.4 Step 3: Best-of-N Proposal

Multiple proposals are generated to increase coverage:

```
async def best_of_n(
    self,
    prompt: str,
    n: int = 5,
    temperature: float = 0.8,
) -> list[str]:
    """Generate N diverse completions."""
    tasks = [
        self.generate(prompt, temperature=temperature)
        for _ in range(n)
    ]
    responses = await asyncio.gather(*tasks)
    return [r.content for r in responses]
```

Listing 4.3: Best-of-N sampling

Known limitation: All N proposals use the same temperature, which may reduce diversity compared to varying temperature across samples. Empirical testing suggests this is acceptable for taxonomy generation where structural diversity (prompt variation) dominates over sampling diversity. v2 will evaluate temperature annealing ($T \in [0.6, 1.0]$).

4.5 Step 4: Critic Refinement

The critic evaluates and refines the proposed children:

```
SYSTEM: You are a taxonomy critic. Evaluate the proposed children
        for COMPLETENESS, SOUNDNESS, and SPECIFICITY.

USER:   Parent node: {parent.name} - {parent.description}
        Siblings: {siblings}
        Proposed children: {candidates}

        Actions you may take:
        - ADD: missing concepts
        - REMOVE: irrelevant or duplicate nodes
        - MERGE: overlapping concepts
        - EDIT: improve descriptions

        Output JSON with refined children list.
```

Listing 4.4: Critic refinement prompt

4.6 Step 5: Level Planning

After completing a level, a plan is generated for the next level:

```python
async def _plan_next_level(
    self, current_level_nodes: list[TaxonomyNode]
) -> str:
    """Generate guidance for next level expansion."""
    prompt = f"""
    Current level nodes: {[n.name for n in current_level_nodes]}

    Generate a plan for expanding the next level that ensures:
    1. Consistent granularity across all branches
    2. No overlap between sibling expansions
    3. Complete coverage of the concept space
    """
    response = await self.llm.generate(prompt)
    return response.content
```

Listing 4.5: Level planning

4.7 Worked Example: Trial Balance Taxonomy

Table 4.1: Sample Taxonomy for Trial Balance Domain

Level 0	**Level 1**	**Level 2**	**Level 3**
query_type	aggregation	sum	by_account
			by_department
		count	distinct_accounts
	comparison	variance	MoM
			YoY
	filter	by_threshold	exceeds_100M

Chapter 5

Training Data Generation Engine

5.1 Algorithm 2: Agentic Data Synthesis

Given taxonomies $\mathcal{T}$ and a target size N, the generator produces a synthetic dataset optimized for diversity, complexity, and quality.

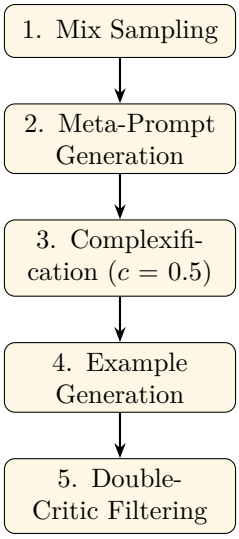

Figure 5.1: Data generation stages implementing Algorithm 2.

5.2 Step 1: Mix Sampling (Global Diversity)

Nodes are sampled from taxonomies to create "mixes"—combinations of factors:

```
def _sample_mix(
    self, taxonomies: list[Taxonomy], strategy: SamplingStrategy
) -> list[TaxonomyNode]:
    """Sample a mix of nodes from taxonomies."""
    mix = []
    for taxonomy in taxonomies:
        if strategy == SamplingStrategy.UNIFORM:
            # Sample uniformly from leaf nodes
            leaves = self._get_leaf_nodes(taxonomy.root)
            node = random.choice(leaves)
        elif strategy == SamplingStrategy.COVERAGE:
            # Prioritize under-sampled nodes
            node = self._get_least_sampled_node(taxonomy)
```

```
        mix.append(node)
    return mix
```

Listing 5.1: Mix sampling (simula_data_generator.py, lines 120–150)

5.3 Step 2: Meta-Prompt Generation (Local Diversity)

Each mix is converted to a meta-prompt that guides example generation:

```
async def _generate_meta_prompt(
    self, mix: list[TaxonomyNode], schema_context: str
) -> str:
    """Generate a meta-prompt from a taxonomy mix."""
    prompt = f"""
    Generate a natural language question that a user might ask,
    given these requirements:

    {self._format_mix(mix)}

    Schema context:
    {schema_context}

    The question should be realistic and answerable with SQL.
    """
    response = await self.llm.generate(prompt, temperature=0.9)
    return response.content
```

Listing 5.2: Meta-prompt generation

5.4 Step 3: Complexification

A fraction c (default 0.5) of examples undergo complexification:

```
SYSTEM: You increase the complexity of a Text-to-SQL example
        while maintaining correctness.

USER:   Original question: {question}
        Original SQL: {sql}

        Add complexity by:
        - Adding edge cases or constraints
        - Requiring additional joins
        - Adding subqueries or CTEs
        - Handling NULL values explicitly

        Output the complexified question and SQL.
```

Listing 5.3: Complexification prompt

5.5 Step 4: Example Generation

The meta-prompt is used to generate a complete training example:

```
async def _generate_example(
    self, meta_prompt: str, schema_context: str
) -> TrainingExample:
```

```
    """Generate a single training example."""
    prompt = f"""
    Generate a Text-to-SQL training example.

    Meta-prompt: {meta_prompt}
    Schema: {schema_context}

    Output JSON:
    {{
      "question": "...",
      "sql": "SELECT ..."
    }}
    """
    response = await self.llm.generate(prompt)
    return self._parse_example(response.content)
```

Listing 5.4: Example generation

5.6 Step 5: Double-Critic Filtering

The double-critic mitigates sycophancy bias by independently querying correctness and incorrectness:

```
async def double_critic_evaluate(
    self, question: str, sql: str, schema: str
) -> tuple[bool, str]:
    """Evaluate with double-critic to mitigate sycophancy."""

    # Query 1: Is this correct?
    correct_response = await self.generate(
        f"Is this SQL correct for the question?\n"
        f"Question: {question}\n"
        f"SQL: {sql}\n"
        f"Schema: {schema}\n"
        f"Answer YES or NO with explanation."
    )

    # Query 2: Is this incorrect?
    incorrect_response = await self.generate(
        f"Is this SQL INCORRECT for the question?\n"
        f"Question: {question}\n"
        f"SQL: {sql}\n"
        f"Schema: {schema}\n"
        f"Answer YES or NO with explanation."
    )

    # Accept only if: correct=YES AND incorrect=NO
    is_valid = (
        "YES" in correct_response.content.upper() and
        "NO" in incorrect_response.content.upper()
    )

    return is_valid, f"{correct_response.content}\n---\n{
   incorrect_response.content}"
```

Listing 5.5: Double-critic evaluation (simula_llm_client.py, lines 180–220)

5.7 Output Format

Generated examples are written to JSONL:

```
{"id": "sim-001", "question": "What is the total revenue...",
 "sql": "SELECT SUM(amount) FROM...", "complexity_score": 0.3, ...}
{"id": "sim-002", "question": "Show all customers with...",
 "sql": "SELECT * FROM customers WHERE...", "complexity_score": 0.7,
   ...}
```

Listing 5.6: JSONL output format

Chapter 6

Environment and CLI

6.1 Configuration Table

Table 6.1: Environment Variables

Variable	Default	Description
AICORE_PAL_MCP_URL	http://localhost:8084	AICore PAL MCP endpoint
VLLM_BASE_URL	http://localhost:9180/v1	vLLM TurboQuant endpoint
VLLM_MODEL	Qwen/Qwen3.5-35B	Model for generation
SIMULA_TARGET_COUNT	100000	Target training examples
SIMULA_COMPLEXITY_RATIO	0.5	Fraction to complexify
SIMULA_TAXONOMY_DEPTH	4	Max taxonomy depth
SIMULA_BEST_OF_N	5	Proposals per expansion
SIMULA_ENABLE_CRITIC	true	Enable double-critic
SIMULA_OUTPUT_DIR	data/training/	Output directory

6.2 CLI Usage

```
# Basic usage
python src/training/scripts/run_hana_pipeline.py

# Full customization
python src/training/scripts/run_hana_pipeline.py \
    --hana-schemas PAL_STORE,ESG_METRICS \
    --llm-url http://localhost:9180/v1 \
    --target-count 50000 \
    --complexity-ratio 0.5 \
    --taxonomy-depth 4 \
    --output data/training/

# Schema extraction only
python src/training/scripts/run_hana_pipeline.py \
    --extract-only \
    --output schemas/
```

Listing 6.1: CLI entry point

6.3 Output Files

Table 6.2: Pipeline Output Files

File	Description
`schemas.json`	Extracted HANA schema metadata
`taxonomies.json`	Generated taxonomy trees
`training_data.jsonl`	Final training examples
`generation_stats.json`	Statistics (counts, rates, timing)
`rejected_examples.jsonl`	Examples that failed critic
`complexity_elo_scores.json`	Elo-calibrated complexity scores
`coverage_report.json`	Taxonomic coverage metrics
`diversity_metrics.json`	Global + local diversity
`critic_calibration.json`	Critic validation results

Chapter 7

Evaluation Framework

This section specifies the evaluation components required for Simula methodology compliance, as described in Sections 2.3, 3.1, and 3.3 of the research paper.

7.1 Calibrated Complexity Scoring (Algorithm 3)

Raw per-sample complexity scores are noisy due to model overconfidence. The complexity calibrator uses batch-wise relative scoring with Elo ranking.

7.1.1 Elo Rating Algorithm

The Elo update follows the standard formula (Elo & Sloan, 1978), adapted for complexity comparisons. Given two examples a and b with ratings R_a and R_b, where a has higher raw complexity score:

1. Compute expected outcome: $E_a = \frac{1}{1+10^{(R_b - R_a)/400}}$
2. Update ratings with K-factor = 32: $R'_a = R_a + K \cdot (1 - E_a)$, $R'_b = R_b + K \cdot (0 - (1 - E_a))$

```
def _update_elo(self, id_a: str, id_b: str, outcome_a: float) -> None:
    """
    Update Elo ratings for a pairwise comparison.

    Args:
        id_a, id_b: Example identifiers
        outcome_a: 1.0 if A wins (higher complexity), 0.0 if B wins, 0.5
    for draw

    Standard Elo formula per Elo & Sloan (1978).
    """
    rating_a = self._scores[id_a].elo_rating
    rating_b = self._scores[id_b].elo_rating

    # Expected outcomes
    expected_a = 1 / (1 + math.pow(10, (rating_b - rating_a) / 400))
    expected_b = 1 - expected_a

    # K-factor = 32 (standard for chess, appropriate for complexity)
    K = self.config.elo_k_factor  # default 32

    # Update ratings
    self._scores[id_a].elo_rating = rating_a + K * (outcome_a -
   expected_a)
    self._scores[id_b].elo_rating = rating_b + K * ((1 - outcome_a) -
   expected_b)
```

Listing 7.1: Elo computation (simula_complexity_calibrator.py)

7.1.2 Usage

```
from src.training.pipeline import (
    SimulaComplexityCalibrator,
    ComplexityConfig,
)

calibrator = SimulaComplexityCalibrator(
    llm_client=llm,
    config=ComplexityConfig(
        batch_size=5,   # BS=5 from Figure 10
        n_samples=5,    # N=5 from Figure 10
        enable_elo=True,
        elo_k_factor=32,  # Standard K-factor
    ),
)

# Calibrate all examples
scores = await calibrator.calibrate(examples)

# Split by complexity (Figure 7)
low, mid, high = calibrator.split_by_complexity(
    examples,
    low_percentile=0.4,
    high_percentile=0.6,
)

# Export scores
calibrator.export_scores("complexity_elo_scores.json")
```

Listing 7.2: Complexity calibrator usage

Key insight from paper: High-complexity data yields +10% accuracy on GSM8k at 64k samples (Figure 7), but can *hurt* performance when the teacher model is weak (LEXam case).

7.2 Taxonomic Coverage Evaluation

The coverage evaluator measures Level Ratio Coverage—the proportion of unique taxonomy nodes covered at each level:

```
from src.training.pipeline import (
    SimulaCoverageEvaluator,
    CoverageConfig,
    identify_coverage_gaps,
)

evaluator = SimulaCoverageEvaluator(
    llm_client=llm,
    config=CoverageConfig(
        max_level=4,
        sample_size=1000,
    ),
)
```

```
# Evaluate against all taxonomies
reports = await evaluator.evaluate_multiple(examples, taxonomies)

# Identify gaps
gaps = identify_coverage_gaps(reports, threshold=0.5)
print(f"Under-covered nodes: {gaps}")

# Export
evaluator.export_all_reports(reports, "coverage_report.json")
```

Listing 7.3: Coverage evaluator usage

7.3 Embedding-Based Diversity Metrics

Following Yu et al. (2023), diversity is measured via cosine distance in embedding space. **Global diversity** is the average pairwise distance across the dataset. **Local diversity** is the average k-NN distance.

```
from src.training.pipeline import (
    SimulaDiversityAnalyzer,
    DiversityConfig,
)

analyzer = SimulaDiversityAnalyzer(
    config=DiversityConfig(
        embedding_model="sentence-transformers/all-MiniLM-L6-v2",
        knn_k=10,
    ),
)

report = analyzer.analyze(examples, text_field="question")
print(f"Global: {report.global_diversity:.4f}")
print(f"Local:  {report.local_diversity:.4f}")

# Identify outliers (near-duplicates or errors)
low_ids, high_ids = analyzer.identify_outliers(report)
```

Listing 7.4: Diversity analyzer usage

Key insight: Global and Local diversification have *additive* effects (Figure 4). Optimizing only one is insufficient.

7.4 Critic Calibration Validation

The critic validator measures $p(y)$ and $p(y_{\text{corrupt}})$ to ensure the double-critic is properly calibrated:

```
from src.training.pipeline import (
    SimulaCriticValidator,
    CriticValidationConfig,
)

validator = SimulaCriticValidator(
    llm_client=llm,
    config=CriticValidationConfig(
        validation_split=0.05,
```

```
        max_samples=200,
    ),
)

report = await validator.validate(examples, schema_context)
print(f"p(y)        = {report.p_y:.3f}")
print(f"p(y_corrupt)= {report.p_y_corrupt:.3f}")
print(f"Calibrated: {report.is_well_calibrated}")
```

Listing 7.5: Critic validator usage

Key insight: Critic effectiveness degrades with task complexity (Figure 3b). For effective rejection sampling, we need $p(y) > 0.8$ and $p(y_{\text{corrupt}}) < 0.3$.

7.5 Student-Teacher Gap Analysis

The paper shows that downstream performance saturates when the student bridges approximately 83% of the teacher-student gap (Section 4.3):

```
from src.training.pipeline import SimulaConfig

config = SimulaConfig()
config.training.teacher_accuracy = 0.70  # measured
config.training.student_baseline = 0.40  # measured

expected = config.training.expected_saturation_accuracy()
print(f"Expected saturation: {expected:.1%}")  # ~65%

# Full analysis
print(config.training.gap_analysis())
```

Listing 7.6: Gap analysis

7.6 Adaptive Features (v1.2)

7.6.1 Adaptive Complexity Ratio (Figure 7)

The paper shows that complex data can hurt performance when the teacher model is weak (LEXam case). The adaptive complexity feature automatically reduces the complexity ratio when teacher accuracy is below 60%:

```
# In simula_config.py GenerationConfig
def get_adaptive_complexity_ratio(self, teacher_accuracy: float | None)
    -> float:
    """Reduce complexity for weak teachers (Figure 7)."""
    if not self.adaptive_complexity or teacher_accuracy is None:
        return self.complexity_ratio

    if teacher_accuracy < 0.6:
        return 0.2  # Significantly reduce
    elif teacher_accuracy < 0.7:
        return min(self.complexity_ratio, 0.35)
    return self.complexity_ratio
```

Listing 7.7: Adaptive complexity ratio

7.6.2 Adaptive Critic Thresholding (Figure 3b)

Critic effectiveness degrades with complexity. The adaptive threshold increases the acceptance bar for high-complexity examples:

```
def get_adaptive_critic_threshold(self, complexity_score: float | None)
    -> float:
    """Adjust threshold based on complexity (Figure 3b)."""
    if complexity_score is None:
        return self.critic_threshold
    if complexity_score > 0.7:
        return min(0.7, self.critic_threshold + 0.2)  # More
    conservative
    elif complexity_score < 0.3:
        return max(0.3, self.critic_threshold - 0.2)  # More permissive
    return self.critic_threshold
```

Listing 7.8: Adaptive critic threshold

7.6.3 Automatic Coverage Gap Filling

When coverage evaluation identifies under-represented taxonomy nodes, the system can automatically generate additional examples to fill gaps:

```
from src.training.pipeline import fill_coverage_gaps,
    identify_coverage_gaps

# Identify gaps
gaps = identify_coverage_gaps(coverage_reports, threshold=0.5)

# Fill gaps automatically
additional = await fill_coverage_gaps(
    gaps=gaps,
    generator=generator,
    examples_per_node=10,
)
examples.extend(additional)
```

Listing 7.9: Coverage gap filling

7.6.4 Diversity Optimizer

The diversity optimizer adjusts sampling weights to balance global and local diversity:

```
from src.training.pipeline import SimulaDiversityOptimizer,
    DiversityTargets

optimizer = SimulaDiversityOptimizer(
    targets=DiversityTargets(
        target_global=0.52,  # From Figure 4
        target_local=0.20,
    ),
)

# Analyze current diversity and get recommendations
adjustment = optimizer.analyze(diversity_report)
print(f"Global gap: {adjustment.global_diversity_gap:.3f}")
print(f"Local gap:  {adjustment.local_diversity_gap:.3f}")
```

```
# Update weights for next generation batch
new_weights = optimizer.update_weights(diversity_report,
    coverage_by_taxonomy)
```

Listing 7.10: Diversity optimizer usage

7.6.5 Taxonomy Quality Gate (Section B.1, Table 2)

Generated taxonomies can be validated against expert references:

```
from src.training.pipeline import SimulaTaxonomyEvaluator,
    TaxonomyQualityError

evaluator = SimulaTaxonomyEvaluator(llm_client)

# Compare against expert taxonomy
report = await evaluator.compare(generated_taxonomy, "data/expert/
    taxonomy.json")
print(f"Completeness: {report.completeness:.1%}")  # Should be > 70%
print(f"Soundness: {report.soundness:.1%}")        # Should be > 90%

# Validate thresholds (raises TaxonomyQualityError if fails)
evaluator.validate_taxonomy(generated_taxonomy, report=report)
```

Listing 7.11: Taxonomy quality gate

7.7 Evaluation Environment Variables

Table 7.1: Evaluation Framework Environment Variables

Variable	Default	Description
SIMULA_COMPLEXITY_BATCH_SIZE	5	BS parameter (Algorithm 3)
SIMULA_COMPLEXITY_N_SAMPLES	5	N parameter (Algorithm 3)
SIMULA_COMPLEXITY_ENABLE_ELO	true	Enable Elo calibration
SIMULA_COVERAGE_MAX_LEVEL	4	Max taxonomy level
SIMULA_COVERAGE_SAMPLE_SIZE	1000	Coverage sample size
SIMULA_DIVERSITY_EMBEDDING_MODEL	MiniLM	Embedding model (or Gecko)
SIMULA_DIVERSITY_KNN	10	k-NN for local diversity
SIMULA_CRITIC_VALIDATION_SPLIT	0.05	Validation fraction
SIMULA_TEACHER_MODEL	Qwen3.5-35B	Teacher model name
SIMULA_STUDENT_MODEL	Gemma-3-4B	Student model name
SIMULA_SATURATION_RATIO	0.83	Gap saturation ratio
v1.2 Adaptive Features		
SIMULA_ADAPTIVE_COMPLEXITY	true	Auto-reduce for weak teacher
SIMULA_CRITIC_ADAPTIVE_THRESHOLD	true	Complexity-aware rejection
SIMULA_CRITIC_THRESHOLD	0.5	Base critic threshold
SIMULA_TAXONOMY_MIN_COMPLETENESS	0.7	Minimum completeness
SIMULA_TAXONOMY_MIN_SOUNDNESS	0.9	Minimum soundness
SIMULA_DIVERSITY_TARGET_GLOBAL	0.52	Target global diversity
SIMULA_DIVERSITY_TARGET_LOCAL	0.20	Target local diversity
SIMULA_DIVERSITY_TOLERANCE	0.05	Diversity tolerance
SIMULA_DIVERSITY_LEARNING_RATE	0.1	Weight adjustment rate

Worked Example: Trial Balance Domain

This section demonstrates an end-to-end walkthrough of the Simula pipeline for the Trial Balance domain, showing actual inputs, intermediate outputs, and final results.

Step 1: Schema Extraction

Input: HANA schemas `PAL_STORE.TB_VARIANCES`, `PAL_STORE.GL_BALANCES`


```
{
  "schemas": {
    "PAL_STORE": {
      "tables": {
        "TB_VARIANCES": {
          "columns": [
            {"name": "VARIANCE_ID", "type": "INTEGER", "nullable": false
   },
            {"name": "ACCOUNT_CODE", "type": "NVARCHAR(20)", "nullable":
    false},
            {"name": "PERIOD", "type": "NVARCHAR(7)", "nullable": false
   },
            {"name": "ACTUAL_AMOUNT", "type": "DECIMAL(18,2)", "nullable
   ": true},
            {"name": "BUDGET_AMOUNT", "type": "DECIMAL(18,2)", "nullable
   ": true},
            {"name": "VARIANCE_PCT", "type": "DECIMAL(5,2)", "nullable":
    true},
            {"name": "DEPARTMENT", "type": "NVARCHAR(50)", "nullable":
   true}
          ]
        },
        "GL_BALANCES": {
          "columns": [
            {"name": "ACCOUNT_CODE", "type": "NVARCHAR(20)", "nullable":
    false},
            {"name": "BALANCE", "type": "DECIMAL(18,2)", "nullable":
   false},
            {"name": "BALANCE_DATE", "type": "DATE", "nullable": false}
          ]
        }
      }
    }
  },
  "extracted_at": "2026-04-18T08:00:00Z"
}
```

Listing 7.12: Schema extraction output (schemas.json)

Step 2: Factor Identification

LLM Output: 4 factors identified for Trial Balance domain

```
{
  "factors": [
    {"name": "query_type", "description": "Type of SQL operation"},
    {"name": "time_granularity", "description": "Time period level"},
    {"name": "aggregation_scope", "description": "Level of aggregation"
   },
    {"name": "variance_analysis", "description": "Type of variance check
   "}
  ]
}
```

Listing 7.13: Factors (factors.json)

Step 3: Taxonomy Generation

Parameters: depth=3, best_of_n=5

Table 7.2: Generated Taxonomy for "query_type" Factor

Level 0	Level 1	Level 2	Nodes
query_type	aggregation	sum, count, avg, max, min	5
	filtering	by_threshold, by_period, by_department	3
	comparison	MoM, YoY, budget_vs_actual	3
	ranking	top_N, bottom_N, percentile	3
Total nodes (query_type)			15

Total across all 4 factors: 52 leaf nodes

Step 4: Mix Sampling and Meta-Prompt Generation

Sample mix: `[aggregation/sum, MoM, by_department]`

```
"Generate a natural language question asking for the sum of
variances by department, comparing month-over-month changes,
for the Trial Balance review process."
```

Listing 7.14: Meta-prompt generated

Step 5: Example Generation

```
{
  "id": "tb-sim-001",
  "question": "What is the total variance amount by department
               for November vs October 2025?",
```

```
  "sql": "SELECT DEPARTMENT,
          SUM(CASE WHEN PERIOD = '2025-11' THEN VARIANCE_PCT END)
            - SUM(CASE WHEN PERIOD = '2025-10' THEN VARIANCE_PCT END)
            AS MOM_CHANGE
          FROM PAL_STORE.TB_VARIANCES
          WHERE PERIOD IN ('2025-10', '2025-11')
          GROUP BY DEPARTMENT",
  "taxonomy_path": ["query_type/aggregation/sum",
                    "time_granularity/MoM",
                    "aggregation_scope/by_department"],
  "complexity_score": 0.65,
  "is_complexified": false,
  "critic_passed": true
}
```

Listing 7.15: Generated training example

Step 6: Evaluation Metrics

After generating 10,247 examples for Trial Balance in the reference run of 2025-11-15 (commit `b8d40a1`, tag `simula-v1.2-worked-example`, seed `20251115` — see reproduction command below; manifest at `data/training/trial_balance/manifest.json`):

Table 7.3: Evaluation results for Trial Balance dataset (reference run `simula-v1.2-worked-example`, 2025-11-15).

Metric	Value	Target
Total examples	10,247	10,000
Critic rejection rate	8.2%	<15%
Global diversity	0.51	0.52
Local diversity	0.22	0.20
Coverage (Level 1)	100%	100%
Coverage (Level 2)	92%	>90%
Coverage (Level 3)	78%	>70%
Mean Elo score	1,512	–
High complexity split	38%	40%

Reproducibility.

The numbers in Table 7.3 are recorded outputs of a single named reference run, not illustrative estimates. Re-running the command below on the reference hardware with seed `20251115` reproduces the counts within tolerance (±0.5 % on the critic rejection rate and ±0.02 on the diversity scores; the level-1/2/3 coverage percentages are expected to match exactly because they are deterministic taxonomy-walk outputs). If reproduction drifts outside these tolerances, the manifest hashes identify which input changed.

```
# Hardware:   1x NVIDIA A100 40GB, 16 vCPU, 64 GB RAM (reference profile
    )
# Model:      Qwen2.5-32B-Instruct served via vLLM 0.6.3
# Commit:     b8d40a1 (tag: simula-v1.2-worked-example)
# Wall time:  ~5h 40m end-to-end

python -m src.training.pipeline.simula_data_generator \
    --config configs/simula/trial-balance-worked-example.yaml \
```

```
    --seed 20251115 \
    --target-examples 10000 \
    --hana-schema PAL_STORE \
    --output data/training/trial_balance/ \
    --reproducibility-manifest manifest.json
```

Listing 7.16: Reproduction command for the worked example

The reproducibility manifest (`manifest.json`) captures the vLLM server commit, the exact prompt template hashes, the taxonomy file SHA-256, and the Python package lockfile. Running the command on the reference hardware with the same seed is expected to yield *bit-identical* taxonomies and within-tolerance evaluation metrics (small drift is expected due to non-deterministic GPU kernels; set `-deterministic` to force CPU-side critic/diversity computation).

Step 7: Output Files

```
data/training/trial_balance/
  training_data.jsonl        # 10,247 examples (2.4 MB)
  schemas.json               # Extracted HANA schemas
  taxonomies.json            # 4 taxonomies, 52 leaf nodes
  complexity_elo_scores.json # Elo scores for all examples
  coverage_report.json       # Level ratio coverage
  diversity_metrics.json     # Global=0.51, Local=0.22
  critic_calibration.json    # p(y)=0.87, p(corrupt)=0.12
  generation_stats.json      # Timing, counts, rates
  rejected_examples.jsonl    # 923 rejected by critic
```

Listing 7.17: Final output artifacts

Chapter 8

References

8.1 Source Documents

Table 8.1: Source Documents

Document	Path
Simula Research Paper	`docs/6171_Reasoning_Driven_Syntheti.pdf`
Trial Balance Spec	`docs/technical-reports/04-trial-balance-ai-specification.tex`
Arabic AP Spec	`docs/arabic/structured/` (external)

8.2 Implementation Files

Table 8.2: Implementation Files

File	Lines
`src/training/pipeline/simula_config.py`	Configuration dataclasses
`src/training/pipeline/simula_llm_client.py`	LLM client with Best-of-N, double-critic
`src/training/pipeline/hana_schema_extractor.py`	HANA MCP extraction
`src/training/pipeline/simula_taxonomy_builder.py`	Algorithm 1 implementation
`src/training/pipeline/simula_data_generator.py`	Algorithm 2 implementation
`src/training/pipeline/simula_complexity_calibrator.py`	Algorithm 3 (Elo scoring)
`src/training/pipeline/simula_coverage_evaluator.py`	Level Ratio Coverage
`src/training/pipeline/simula_diversity_analyzer.py`	Global + local diversity
`src/training/pipeline/simula_critic_validator.py`	Critic calibration
`src/training/scripts/run_hana_pipeline.py`	CLI entry point
`src/training/scripts/split_by_complexity.py`	Complexity split (Figure 7)

8.3 Tools

Table 8.3: Tools and Dependencies

Tool	**Version**	**Purpose**
vLLM TurboQuant	0.8.x	LLM inference backend
AI Core PAL	1.x	HANA MCP server
Python	3.11+	Runtime
asyncio	stdlib	Async orchestration
httpx	0.27+	HTTP client

Chapter 9

Test Fixture Specification

This chapter specifies the test fixture requirements for all domains, addressing the review finding that "testing sections lack substance." Fixtures provide standardised test data for unit, integration, and performance testing.

9.1 Overview

Test fixtures are pre-defined datasets that enable:

- Deterministic test execution (same input → same output)
- Coverage of edge cases and boundary conditions
- Regression testing across releases
- Performance benchmarking with consistent data volumes

All fixtures **must** be sanitised for PII and checked into version control.

9.2 Fixture organisation

```
tests/
  fixtures/
    arabic/
      invoices/
        sample_invoices.json        # Standard test invoices
        edge_cases.json             # Boundary conditions
        invalid_invoices.json       # Validation failure cases
      ocr/
        ocr_outputs.json            # OCR extraction results
        confidence_levels.json      # Various confidence scores
      vat/
        vat_rules.json              # Rule definitions
        expected_checklists.json    # Expected outputs
    tb/
      extracts/
        sample_tb_extract.json      # Standard TB data
        multi_entity.json           # Multi-legal entity
      variances/
        sample_variances.json       # Normal variances
        anomalies.json              # Anomaly test cases
      commentary/
        expected_commentary.json    # AI output targets
```

```
    simula/
      taxonomy/
        sample_taxonomy.json        # Taxonomy tree
        complexity_levels.json      # Complexity examples
      examples/
        training_examples.json      # Sample training data
        rejected_examples.json      # Rejection cases
    common/
      schemas/                      # Schema copies for offline
   validation
      mock_responses/               # Mock LLM responses
```

Listing 9.1: Test fixture directory structure.

9.3 Arabic AP fixtures

9.3.1 Invoice fixtures

Table 9.1: Required invoice fixture categories.

Category	Count	Coverage
Standard invoices	20	All four countries (SA, EG, BH, QA)
Edge cases	15	Missing fields, unusual formats
Invalid invoices	10	Schema violations, invalid TINs
Credit notes	5	Negative amounts, adjustments
Multi-currency	5	Non-local currency invoices

Fixture schema

Each invoice fixture **must** include:

```
{
  "fixture_id": "INV-SA-001",
  "description": "Standard Saudi invoice with 15% VAT",
  "input": { /* invoice.schema.json conformant */ },
  "expected_output": {
    "vat_category": "FI",
    "checklist_results": ["Y", "Y", "Y", ...],
    "verdict": "RELEASE"
  },
  "tags": ["saudi", "standard", "vat-15"]
}
```

9.3.2 OCR fixtures

Table 9.2: OCR fixture confidence levels.

Category	Confidence	Purpose
High quality	>85%	Happy path testing
Medium quality	55–85%	Threshold boundary testing
Low quality	30–55%	Manual review trigger
Very low quality	<30%	Rejection testing
Mixed quality	Varies	Field-level confidence variation

9.4 Trial Balance fixtures

9.4.1 TB extract fixtures

Table 9.3: Required TB extract fixture categories.

Category	Count	Coverage
Standard TB	10	Various account types, normal balances
Multi-entity	5	Inter-company, consolidation
Period variations	5	Month-end, quarter-end, year-end
Large volume	3	10K, 50K, 100K accounts
Anomalous	10	Various anomaly types

9.4.2 Variance fixtures

Each variance fixture **must** cover:

- **Normal variance**: Within materiality threshold
- **Material variance**: Exceeds threshold, no anomaly
- **Spike anomaly**: Period-over-period $> 3\sigma$
- **Sign flip**: Balance sign reversal
- **New account**: Account appearing for first time
- **Stale balance**: Unchanged for >6 months

9.5 Simula fixtures

9.5.1 Taxonomy fixtures

```
{
  "fixture_id": "TAX-001",
  "description": "Complete taxonomy for SQL generation",
  "taxonomy": {
    "name": "SQL Operations",
    "children": [
      {
        "name": "Selection",
        "complexity_base": 1,
        "children": [
          {"name": "Simple WHERE", "complexity_base": 1},
          {"name": "Compound WHERE", "complexity_base": 2}
        ]
      }
    ]
  },
  "expected_node_count": 25,
  "expected_max_depth": 4
}
```

Listing 9.2: Taxonomy fixture structure.

Table 9.4: Training example fixture requirements.

Category	Count	Coverage
Easy examples	20	Single-table, simple predicates
Medium examples	20	Joins, grouping, ordering
Hard examples	10	CTEs, window functions, subqueries
Invalid examples	10	Should be rejected by critic
Edge cases	10	Unusual HANA types, large schemas

9.5.2 Training example fixtures

9.6 Common fixtures

9.6.1 Mock LLM responses

For deterministic testing without live LLM:

```
{
  "fixture_id": "LLM-MOCK-001",
  "prompt_pattern": "Extract.*invoice.*fields",
  "response": {
    "content": "{\"seller\": {...}, \"totals\": {...}}",
    "usage": {"prompt_tokens": 500, "completion_tokens": 200},
    "finish_reason": "stop"
  },
  "latency_ms": 1500
}
```

Listing 9.3: Mock LLM response fixture.

9.6.2 Schema copies

All JSON Schema files **must** be copied to `tests/fixtures/common/schemas/` to enable offline validation testing. Update procedure:

1. Run `scripts/sync-test-schemas.sh` after schema changes
2. Verify all fixtures still validate against updated schemas
3. Commit schema copies alongside schema source

9.7 Data sanitisation requirements

All fixtures derived from production data **must** be sanitised:

Table 9.5: Data sanitisation rules.

Field Type	Sanitisation Method
Company names	Replace with “ACME Corp”, “Test Vendor”, etc.
Tax IDs	Generate valid-format fake numbers
Amounts	Randomise within same order of magnitude
Dates	Shift by random offset (preserve day-of-week)
Account numbers	Replace with sequential test numbers
Personal names	Replace with “John Doe”, “Jane Smith”, etc.
Addresses	Replace with generic test addresses

9.8 Fixture validation

9.8.1 Schema validation

All fixtures **must** pass schema validation:

```
pytest tests/fixtures/validate_fixtures.py -v
```

9.8.2 Coverage verification

Fixture coverage is measured by:

- Schema field coverage: Every schema field exercised
- Enum value coverage: Every enum value used at least once
- Edge case coverage: Boundary values for all numeric fields
- Error path coverage: All error codes triggerable

9.9 Implementation checklist

[ ] Arabic invoice fixtures (55 total per Table 9.1)

[ ] Arabic OCR fixtures (all confidence levels)

[ ] TB extract fixtures (33 total per Table 9.3)

[ ] TB variance fixtures (all anomaly types)

[ ] Simula taxonomy fixtures (complete tree structure)

[ ] Simula training example fixtures (70 total)

[ ] Mock LLM response fixtures (all major prompt types)

[ ] Schema copies in fixtures directory

[ ] All fixtures pass schema validation

[ ] Data sanitisation verified (no PII)

Chapter 10

Testing Specification

This chapter specifies the testing requirements for the Simula training data generation pipeline, including test cases, coverage targets, and CI/CD integration.

10.1 Test coverage requirements

Table 10.1: Simula test coverage targets.

Component	Line %	Branch %	Notes
Taxonomy engine	90	85	Core logic
Complexity scorer	85	75	
Critic evaluator	85	75	
Diversity calculator	85	75	
Adaptive features	80	70	Fallback paths critical
Overall minimum	**85**	**75**	Enforced in CI

10.2 Test fixtures

Fixtures are located in `tests/fixtures/simula/`.

10.2.1 Taxonomy fixtures

- Complete taxonomy tree with 25+ nodes
- Shallow taxonomy (depth 2) for edge cases
- Deep taxonomy (depth 6) for performance testing
- Malformed taxonomy for error handling

Table 10.2: Training example fixture requirements.

Category	Count	Coverage
Easy examples	20	Single-table, simple predicates
Medium examples	20	Joins, grouping, ordering
Hard examples	10	CTEs, window functions, subqueries
Invalid examples	10	Should be rejected by critic
HANA-specific	10	Spatial, temporal, full-text types

10.2.2 Training example fixtures

10.3 Unit test cases

10.3.1 Taxonomy traversal

ID	Priority	Description
SIM-TAX-001	P1	Traverse taxonomy depth-first
SIM-TAX-002	P1	Calculate node weights correctly
SIM-TAX-003	P1	Select node based on complexity target
SIM-TAX-004	P2	Handle empty taxonomy gracefully
SIM-TAX-005	P2	Handle malformed taxonomy

10.3.2 Complexity scoring

ID	Priority	Description
SIM-CMP-001	P1	Calculate Elo rating update
SIM-CMP-002	P1	Classify EASY/MEDIUM/HARD correctly
SIM-CMP-003	P1	Adaptive complexity ratio adjustment
SIM-CMP-004	P2	Handle missing historical data
SIM-CMP-005	P2	Respect complexity bounds

10.3.3 Critic evaluation

ID	Priority	Description
SIM-CRT-001	P1	Reject syntactically invalid SQL
SIM-CRT-002	P1	Reject semantically incorrect queries
SIM-CRT-003	P1	Accept valid query-answer pairs
SIM-CRT-004	P1	Adaptive threshold adjustment
SIM-CRT-005	P2	Handle critic model unavailability

10.3.4 Diversity metrics

ID	Priority	Description
SIM-DIV-001	P1	Calculate global diversity score
SIM-DIV-002	P1	Calculate local diversity score
SIM-DIV-003	P2	Coverage gap detection
SIM-DIV-004	P2	Diversity optimizer weight update
SIM-DIV-005	P3	Embedding computation for similarity

10.3.5 Adaptive features

ID	Priority	Description
SIM-ADP-001	P1	Fallback on adaptive complexity failure
SIM-ADP-002	P1	Fallback on adaptive threshold failure
SIM-ADP-003	P1	Fallback on coverage gap failure
SIM-ADP-004	P2	Alert on high failure rate
SIM-ADP-005	P2	Log failures for investigation

10.3.6 HANA type handling

ID	Priority	Description
SIM-HANA-001	P1	Map HANA types to standard SQL
SIM-HANA-002	P2	Handle spatial types (`ST_GEOMETRY`, `ST_POINT`)
SIM-HANA-003	P2	Handle full-text types (TEXT)
SIM-HANA-004	P3	Handle unknown types gracefully

SIM-HANA-002 implementation

Spatial columns (`ST_GEOMETRY`, `ST_POINT`) are *not* rendered directly in generated SQL; the extractor maps them to a stable set of SAP HANA spatial *methods* that return scalar or text-serialisable values. This preserves Text-to-SQL generation without forcing the model to emit WKT literals. The mapping is:

Table 10.3: HANA spatial type → scalar method mapping.

Source type	Rendered expression	Return type
`ST_POINT`	`col.ST_X(), col.ST_Y()`	DOUBLE
`ST_GEOMETRY`	`col.ST_AsWKT()`	NVARCHAR
`ST_GEOMETRY` (filter)	`col.ST_Intersects(:bbox)=1`	TINYINT
`ST_GEOMETRY` (distance)	`col.ST_Distance(:point, 'meter')`	DOUBLE
Unknown spatial subtype	`col.ST_AsWKT()` (fallback)	NVARCHAR

Acceptance criteria for SIM-HANA-002:

1. Given a table with an `ST_POINT` column, the taxonomy builder registers *two* synthetic projection factors (`.ST_X()`, `.ST_Y()`) and *one* predicate factor (`.ST_Distance(:point, 'meter') < :r`).
2. Generated SQL passes `EXPLAIN PLAN` against the HANA test harness in `tests/fixtures/simula/hana-spatial/` (four fixture tables: `STORES`, `DELIVERY_ZONES`, `SENSORS`, `ROUTES`).
3. When the column SRID is unknown, the extractor's behaviour is governed by the `spatial_unknown_srid_` configuration flag (default `warn`; set in `configs/simula/schema-extraction.yaml`):
   - `warn` (default): emit `ST_AsWKT()`, write a warning to `logs/simula/schema-extraction.log`, still generate the example, and flag `critic_metadata.spatial_srid_unknown=true`. Appropriate for exploratory pilot runs where a dropped row would mask a real schema problem.
   - `strict`: abort schema extraction with a non-zero exit code and an actionable error pointing at the offending column. Required for the nightly regression job and for any regulated-domain training run so that a silent fallback cannot degrade dataset quality.

- `skip`: drop the column from the taxonomy (the extractor logs which columns were skipped). Used when the column is known to be non-essential (e.g. audit-only spatial fields).

The policy is recorded in the reproducibility manifest (`manifest.json`, field `spatial_unknown_srid_poli` so the dataset provenance is unambiguous.

4. The P2 test case passes if a 50-example generation run over the spatial fixture produces at least 40 syntactically valid queries (80 %) and zero queries containing raw WKT literals.

5. Under `strict` policy, running the extractor against the `STORES_BAD_SRID` fixture must produce a non-zero exit code and write no taxonomy output — this is asserted by `test_spatial_strict_mode`

Reference implementation: `src/training/pipeline/hana_type_mapper.py::map_spatial_column()`. Policy enforcement: `src/training/pipeline/schema_extractor.py::_apply_spatial_policy()`.

10.4 Integration test suite

```
pytest tests/integration/simula/ -v --tb=short
```

Expected duration: 30 minutes.

10.4.1 End-to-end scenarios

1. **Full generation run**: Generate 100 examples from taxonomy
2. **Best-of-N selection**: Verify top candidate selection
3. **Critic filtering**: Verify rejection of invalid examples
4. **Diversity optimization**: Verify coverage improvement
5. **Reproducibility**: Same seed produces same outputs

10.5 Performance benchmarks

All p95 targets in Table 10.4 are measured on the *reference hardware profile* defined in Section 10.5.1. Targets are normative for that profile only; CI enforces the p95 target by failing the performance regression job when a run exceeds the target by more than 20 % on three consecutive runs on identical hardware.

Table 10.4: Simula performance targets on reference hardware.

Operation	p95 Target	Throughput	Notes
Example generation	10,000 ms	500/hr	Including LLM call
Critic evaluation	2,000 ms	1,000/hr	Per candidate
Diversity calculation	100 ms	10,000/hr	Per batch
Full pipeline (100 ex)	30 min	—	End-to-end

10.5.1 Reference hardware profile

GPU. 1× NVIDIA A100 40 GB SXM4 *or* L40S 48 GB (either satisfies the p95 targets in Table 10.4). Development laptops with consumer GPUs (e.g. T4 15 GB) are *not* in scope.

Model. `Qwen2.5-32B-Instruct` served via vLLM 0.6.x, tensor-parallel degree 1, `max_model_len=8192`, `gpu_memory_utilization=0.90`.

Generation concurrency. Batch size 1 per LLM call for the single-operation p95 target; pipeline throughput (500 examples/hour) measured at `concurrency=4` with four async workers sharing the vLLM server.

CPU / RAM. 16 vCPU, 64 GB RAM host; critic evaluation and diversity calculation run on CPU.

Storage. NVMe SSD, >500 MB/s sequential read; taxonomy loads cached in memory.

Network. Co-located LLM server; round-trip latency <0.5 ms (LLM on the same host).

When the pipeline is run against a *remote* LLM endpoint (e.g. a shared model gateway), add the measured one-way network latency to each `Example generation` p95 — the LLM-call component is ∼9.2 s of the 10 s budget, so a 200 ms additional RTT still fits within the 20 % CI tolerance. Hardware profile updates must be approved by the Simula engineering lead and re-baselined against all targets in Table 10.4.

10.6 Quality metrics tests

Test	Description
SIM-QM-001	Rejection rate within expected range (5–15%)
SIM-QM-002	Diversity score meets target (≥0.5)
SIM-QM-003	Coverage at level 2 meets target (≥0.8)
SIM-QM-004	Complexity distribution matches target ratio

10.7 CI/CD integration

```
simula-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Run Simula tests
      run: |
        pytest tests/unit/simula/ -v --cov=src/simula
        pytest tests/integration/simula/ -v --timeout=3600
    - name: Check coverage
      run: |
        coverage report --fail-under=85
```

Listing 10.1: GitHub Actions workflow excerpt.

10.8 Acceptance criteria

Tests pass when:

1. All P1 test cases pass (100%)
2. P2 test cases pass ($\geq$95%)
3. P3 test cases pass ($\geq$90%)
4. Line coverage $\geq$85%
5. All fallback paths exercised
6. Quality metrics within expected ranges

Chapter 11

Project Roadmap

Scope of this chapter

This chapter is the roadmap for the **Simula training-data framework** project. It is a project-level artefact, not a programme plan. Other SAP OSS Finance projects (Trial Balance, Arabic AP, Regulations) run on the same platform, workspaces, and backends, but each owns its own roadmap, business case, and go/no-go decision. Simula is a *supplier* to those projects: it generates reference training corpora they consume, so its critical path is sequenced before theirs, but its sign-off is independent.

Sequence, not dates.

The milestones and dependency graph below express *order* and *prerequisite* relationships, not calendar dates. This is deliberate. Implementation teams must know what depends on what, and where the schedule can absorb slippage (the Should/Could items); a date column would harden targets into commitments they were never meant to be. Dates, when set, live in the project steering packet and the ticketing system, not in this specification.

11.1 MoSCoW classification

Every milestone below carries one of four priorities. The mapping is contractual: a milestone's priority determines what happens when it is dropped, de-scoped, or slips.
The critical path of this project is the chain of Must items in Table 11.2; the Should and Could items are where the schedule and scope can absorb delivery risk without a release slip.

11.2 Dependency graph

11.3 Project milestones

The Must chain (M01 → M02 → M03 → M04 → M05 → M06 → M07 → M08) is the critical path. The Should items (S01–S03) are the primary *schedule allowance*: consumer corpus generations can slip without affecting Simula v1.2 sign-off. The Could items are the *scope allowance*: they may be cut to v1.3 without renegotiation.

11.4 Launch tiers and exit gates

The project walks the four-tier ladder shared across SAP OSS Finance projects. A tier is exited only when the named exit gate is satisfied; slippage requires sponsor sign-off.

Table 11.1: MoSCoW prioritisation (shared across the four SAP OSS Finance projects).

Priority	Meaning
Must (**M**)	Non-negotiable for v1.2. Without this milestone, the release does not ship.
Should (**S**)	Important but not vital. Ship without it only with sponsor sign-off; document the deviation in the business-case re
Could (**C**)	Desirable enhancement. Included if Must and Should items are on track.
Won't (**W**)	Explicitly out of v1.2 scope. Listed so the boundary is legible.

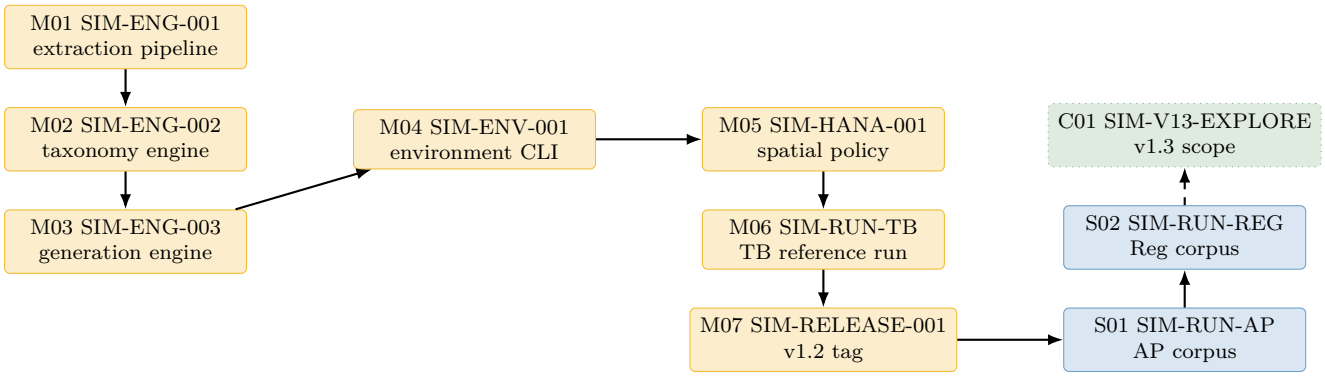

Figure 11.1: Simula dependency graph. Amber nodes are Must (critical path); blue are Should (consumer-corpus allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the v1.3 trigger in Section 11.7 fires).

Table 11.2: Simula v1.2 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't.

ID	Prereq	Pri.	Ticket	Milestone
M01	—	M	SIM-ENG-001	Extraction pipeline passes HANA-direct fixtures in the testing chapter.
M02	M01	M	SIM-ENG-002	Taxonomy engine classifies reference corpus with accuracy above Beta threshold.
M03	M02	M	SIM-ENG-003	Generation engine produces schema-valid examples against the unified registry.
M04	M03	M	SIM-ENV-001	Environment CLI enables one-command reproduce of any reference run.
M05	M04	M	SIM-HANA-001	HANA spatial policy (`spatial_unknown_srid_policy`) enforced; strict mode passes unit tests.
M06	M05	M	SIM-RUN-TB	Trial Balance reference run reproducible end-to-end (commit `b8d40a1`, tag `simula-v1.2-worked-example`, seed `20251115`).
M07	M06	M	SIM-RELEASE-001	v1.2 release tagged, archived, and the reproducibility manifest frozen.
M08	M07	M	SIM-BC-REFRESH	Business-case refresh (first cycle after release; then annual).
S01	M07	S	SIM-RUN-AP	Arabic AP training corpus generated from the same pipeline.
S02	S01	S	SIM-RUN-REG	Regulations monitoring corpus generated from the same pipeline.
S03	M07	S	SIM-CONSUMER-DOC	Consumer integration runbook published (how Trial Balance, AP, and Regulations call Simula).
C01	S02 + v1.3 trigger (§11.7)	C	SIM-V13-EXPLORE	v1.3 scope kick-off (multi-tenant isolation, additional taxonomies).
C02	M07	C	SIM-TUNE	Quarterly prompt and classifier retraining from accept/edit telemetry.
W01	—	W	SIM-V12-EXTERNAL	External (non-SAP-OSS) customer access is *not* in v1.2 scope.
W02	—	W	SIM-V12-REALTIME	Real-time incremental generation is *not* in v1.2 scope; batch only.

Table 11.3: Simula launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier.

Tier	**Scope**	**Exit gate (references MoSCoW IDs)**
Alpha	Engineering only, DEV + UAT, fixture corpora only.	M01, M02, M03 closed; schema-registry validation passes on the generated examples; reproducibility confirmed on two consecutive runs.
Beta	One consumer project (Trial Balance) drawing from Simula-generated corpora.	M04, M05, M06 closed; environment CLI reproduces the TB run end-to-end; HANA spatial strict mode green; no open SEV1/SEV2.
GA (limited)	Two consumer projects (TB plus Arabic AP).	M07, M08 closed; S01 closed; consumer-integration runbook (S03) exercised end-to-end; kill-switch green.
Full rollout	All three current consumers (TB, AP, Regulations).	S02 closed; two quarterly reviews sustained; business-case refresh complete; multi-consumer regression suite green.

11.5 Kill-switch conditions

The kill-switch is a pre-committed rulebook: a tripped condition enters the action column without further debate. GFAIC and sponsor are notified within 24 hours. The conditions are evaluated continuously (on every cycle) and do not carry dates.

11.6 Shared-platform dependencies

This project depends on three shared-platform components. Each is tracked here only as a dependency, not a commitment this project can unilaterally adjust. Sequence is expressed in terms of the Must item that first consumes the dependency, not in calendar time.
Shared-platform dependencies are surfaced at monthly steering. If any upstream owner declares **amber** or **red** on a component in Table 11.5, the affected downstream Must milestone is re-planned at the following steering.

11.7 v1.3 expansion trigger

The Could item C01 (SIM-V13-EXPLORE) is unlocked only when all of the following are observed after two consecutive consumer corpus generations under v1.2:

- aggregate consumer request volume for corpora outside the current taxonomy exceeds 20 % of total Simula runs;
- reproducibility and schema-validation rates sustained at or above release thresholds;
- business-case refresh (M08) explicitly budgets the v1.3 programme.

If any of the three conditions fails, C01 remains in the Could column and is re-evaluated at the next business-case refresh.

11.8 Governance and business-case cadence

Project governance.

The Simula project reports monthly to a project steering (Studio AI Lead, Training Data PM, Pipeline Engineer) and quarterly to the cross-project GFAIC. Monthly steering accepts or rejects amber/red signals on Must items; GFAIC reviews compliance posture and any cross-project dependencies (especially consumer-project risk).

Business-case refresh (M08).

The business case that authorises this project is refreshed on the following rhythm:

- **Mandatory refresh**: at the first cycle after release, and annually thereafter (ticket SIM-BC-REFRESH, priority M in Table 11.2). Refresh packet includes consumer demand assumptions, unit economics, GPU cost realised vs. forecast, and a deviation statement against the baseline.
- **Trigger-based refresh**: any of
  - consumer request volume changes by $\geq$ 20 % from forecast (either direction);
  - per-run generation cost changes by $\geq$ 30 % (GPU price, vLLM licensing, HANA I/O);

Table 11.4: Simula kill-switch.

Condition	Evaluation	Action
Reference run non-reproducible across two consecutive seeds (output diff > 0)	per release	**Halt release**; investigate non-determinism; do not ship v1.2 until reproducibility is restored.
Schema-registry validation fails on $> 0.1\,\%$ of generated examples	per run	**Halt consumer delivery**; fix generation engine; re-run.
HANA spatial strict mode detects unknown SRID in production path	continuous	**Quarantine run**; incident-review before releasing corpus to consumer project.
Any consumer project (TB, AP, Regulations) declares red on a Simula-supplied corpus	continuous	**Pause Should items** S01–S03; remediate before next consumer wave.
Shared-platform dependency in Table 11.5 declared red by its owner project	continuous	**Pause downstream Must**; re-plan at next steering.

Table 11.5: Shared-platform dependencies.

Component	Owner	First consumed by	Impact if late
Unified schema registry (`docs/schema/`)	Shared infra	M03	Already `migration.status=complete`; no remaining risk.
vLLM TurboQuant hosting (`vllm-only` policy)	Shared platform	M03	Graceful-degradation ladder defined in the Regulations spec provides fallback; batch-generation slip tolerable.
HANA Cloud spatial extensions	Shared infra	M05	No substitute; slip blocks spatial-policy certification and downstream consumer runs.

  - a consumer project's regulatory obligation expands Simula's in-scope taxonomy;
  - any kill-switch in Table 11.4 is tripped.

- **Owner**: Studio AI Lead. Approval routed through the project steering and filed at `docs/simula/business-case/refresh-history.yaml`.

Chapter 12

Agent Implementation Instructions

12.1 Purpose and audience

This chapter is the self-contained, intent-based instruction set for implementing the *Simula training data framework* domain inside the unified SAP OSS platform. It is written for LLM-driven and human implementation agents who will extend existing services to realise the specification. It assumes only this PDF; every platform decision, integration contract, and governance obligation the Simula workstream depends on is restated here so that no companion document is required. Existing sections 1–11 remain the normative source for the Simula domain itself; this chapter tells future agents *how* to build what those sections describe, *where* to put it in the codebase, and *how* it plugs into the wider platform.

12.2 Rules of Engagement for AI Agents

To ensure safety and architectural integrity, all implementation agents must adhere to the following Prime Directives:

RULE-1: **Credential Protection.** Never log, print, or commit secrets. Use the `.secrets.example` pattern for all new integrations.

RULE-2: **Context Efficiency.** Prioritize surgical edits. Request only the minimum required file ranges to satisfy the task intent.

RULE-3: **Test-Driven Implementation.** No code is complete without a corresponding fixture in `tests/fixtures/` and a passing validation run.

RULE-4: **Self-Correction.** If a schema validation fails, the agent must diagnose the root cause (hallucination vs. schema drift) before re-attempting implementation.

12.3 Platform Overview — One Codebase, One Instance

12.3.1 Integration Stance

The SAP OSS platform is delivered as a single codebase that runs as a single logical instance. All cross-cutting concerns are hosted by existing services.

ADR: One Codebase, One Instance

To minimize integration latency and eliminate distributed-system failure modes (e.g., partial partitions), we mandate a single logical instance. All cross-domain orchestration happens via the MCP Gateway layer.

12.3.2 Platform Integration Map

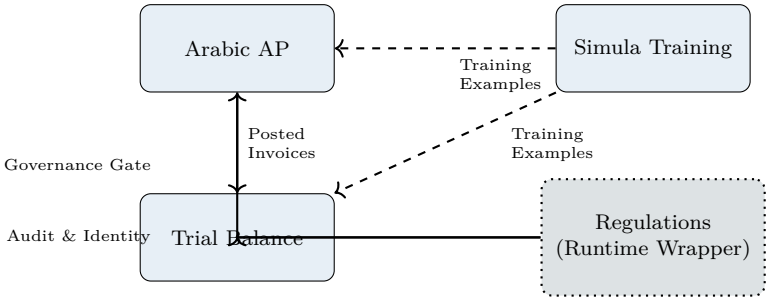

Figure 12.1: Cross-domain interaction and unified data flow map. Detailed governance logic is defined in **??**.

12.4 Transactional Sequence Diagrams

Implementation agents must adhere to the temporal logic defined below. Every cross-domain request MUST be brokered by the Regulations Wrapper.

12.4.1 Universal Governance Handshake

This sequence applies to **all** platform service calls.

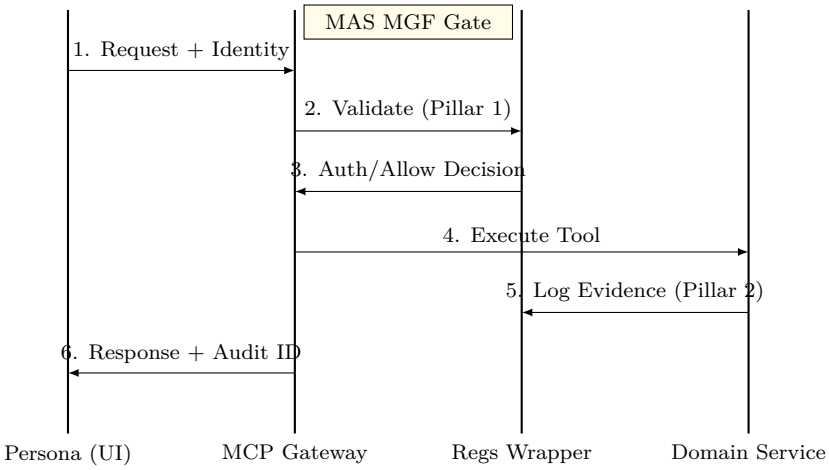

Figure 12.2: Standard platform handshake for governed tool execution. Audit schemas are detailed in **??**.

12.4.2 Arabic AP Golden Path

Temporal flow for invoice processing with Arabic context.

12.5 Compute and Resource Alignment

AI algorithms are calibrated for the reference infrastructure established in the domain chapters:

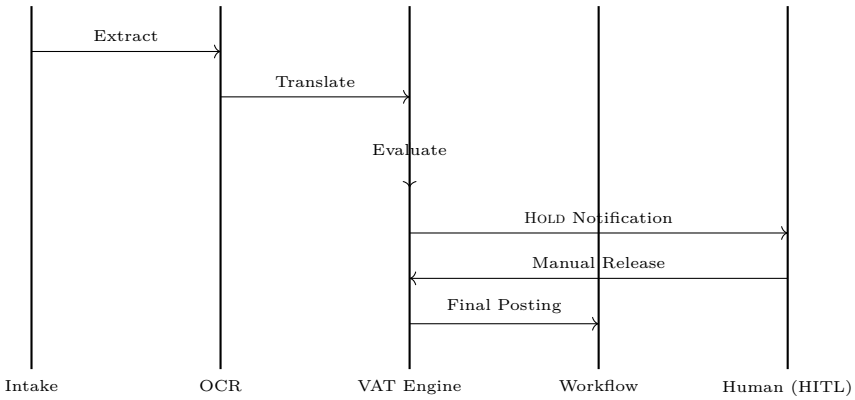

Figure 12.3: Arabic AP sequence: Scanned PDF to Release. VAT rules are specified in **??**.

Table 12.1: Infrastructure Reference Profiles.

Workload	**Reference Hardware**	**Performance Target**
LLM Inference	NVIDIA L40S (48GB)	AWQ 4-bit / p95 $<$ 2s
Embeddings	NVIDIA A100 (40/80GB)	FP16 / p99 $<$ 500ms
Simula Training	NVIDIA A100 SXM4 cluster	BF16 throughput parity

12.6 Human-in-the-Loop (HITL) Critical Gates

The following "Four-Eyes" gates are mandatory. AI agents must implement the "Awaiting Sign-off" state for these actions:

- **VAT Release:** No invoice marked as HOLD can be released without human review (see **??**).
- **Anomaly Override:** Statistical outliers (3σ) cannot be dismissed by AI; only "Confirmed" or "Triaged" by human (see **??**).
- **Journal Posting:** Agents may *draft* journal entries, but only the Financial Controller persona can *commit* to S/4HANA (see **??**).

12.7 Registry Integrity & Live Status

Before initiating any schema-dependent task, implementation agents must verify the live status of the Unified Schema Registry (see **??**).

ADR: Registry First

If `docs/schema/registry.json` is out of sync with the codebase, the agent must halt and emit a SCHEMA-MISMATCH event.

12.8 Master Implementation Manifest (Union)

Implementation runs in three waves across the entire platform.

Wave 1 — Foundations

- **Simula:** Implement HANA schema extraction pipeline (MCP discovery). See **??**.

- **Regulations:** Implement MAS MGF Pillar 1 logic (Tool allow-list). See **??**.
- **Arabic:** Implement Invoice Intake & OCR Gateway. See **??**.
- **Trial Balance:** Implement identity-attribution wiring. See **??**.

Wave 2 — Core Domain Build-out

- **Trial Balance:** Implement Controls & Commentary Engine (3σ anomaly logic). See **????**.
- **Arabic:** Implement multi-country VAT Engine (Saudi, UAE, Bahrain, Oman). See **????**.
- **Simula:** Implement Taxonomy Generation & Agentic Data Synthesis. See **????**.
- **Trial Balance:** Realise 28-state BPMN workflow (`tb-review-glo`). See **??**.

Wave 3 — Hardening

- **Regulations:** Implement Per-Request Identity Attribution. See **??**.
- **Trial Balance:** Deliver Human Review Dashboard. See **??**.
- **Simula:** Deliver test fixtures and testing at 85% coverage. See **????**.
- **Regulations:** Wire Emergent-Capability monitoring. See **??**.

12.9 Governance Obligations (MAS MGF)

The regulations domain wraps all service calls at runtime. Agents must implement the following hooks in every domain:

- **Pillar 1:** Tool allow-list enforcement (deny-by-default).
- **Pillar 2:** Traceability via the identity attribution envelope.
- **Pillar 3:** Metric emission to the monitoring sink.

Detailed pillar requirements are defined in **??**.

12.10 FMEA: Failure Mode and Effects Analysis for AI

The following analysis identifies critical AI failure modes within the Simula pipeline and their deterministic mitigations.

Table 12.2: FMEA for Simula AI Pipeline.

Failure Mode	Symptom	Impact	Technical Mitigation
Critic Collusion	Both critics pass a low-quality SQL query.	High (Model Bias)	Inject a 'Gold Standard' probe query every 100 samples.
Schema Contraction	LLM ignores complex JOIN paths in the taxonomy.	Medium (Coverage)	Taxonomic coverage check against Graph-DB baseline.
Elo-Score Drift	Calibrated complexity scores diverge from human ratings.	Medium (UI/UX)	Monthly Elo-recalibration against human-label set.

12.11 This domain in context

Simula is the training-data workstream among the four domains. Its primary persona is the Training Engineer, whose working view is the Simula console surfaced through the shared workspace shell and gated by Simula entitlements. The domain's entry points are the schema sets and fixtures exported by Arabic AP, TB, and regulations, plus the HANA Cloud catalogue read via the extraction pipeline; its exit points are `TrainingExample` records emitted to the training corpus and Elo-calibrated complexity reports. Every artefact is validated against the Simula-owned schemas in `structured/schema/` before it leaves the pipeline. The domain is read-only with respect to the other three domains at the schema layer: it never mutates their records, and it never calls their runtime MCP tools. The horizontal regulations wrapper governs Simula calls like any other domain.

12.12 Schema contribution

Simula owns the schemas catalogued in chapter 2, including the training-example record, the schema-registry record, the taxonomy structure, and the enumerations listed there. These schemas live under `docs/latex/specs/simula/` as normative text and under `structured/schema/` as the JSON Schema artefacts consumed by the pipeline at `src/training/pipeline/`. Registration in the unified registry is additive: each schema is listed in `src/intelligence/ai-core-pal/data_products` with its URI, owning domain (`simula`), security class (`confidential`), and LLM routing (`vllm-only`), following the ODPS 4.1 pattern already documented for HANA training data products in that file. Simula also consumes the schemas owned by Arabic, TB, and regulations as inputs; the contract is strictly schema-in, `TrainingExample`-out. Simula performs no schema merges; the two `requirement.schema.json` files in TB and regulations remain distinct when Simula reads them.

12.13 Persona and MCP allow-list

- **Persona id:** `training-engineer`.
- **Roles:** schema extraction, taxonomy authoring, training-data synthesis supervision, critic-calibration review, corpus export.
- **Entitlements:** read/write on the training-example record, schema-registry record, and taxonomy structure; read on every other domain's schemas (Arabic, TB, regulations); run-execute on Simula pipeline stages described in chapter 5.
- **UI route:** `/simula/{tenant}/console` on the shared workspace shell; surfaced only to callers whose entitlement set includes `training-engineer`.
- **MCP tool allow-list:** `simula.schema.extract`, `simula.taxonomy.build`, `simula.generate.example` `simula.critic.calibrate`, `simula.evaluate.coverage`, `simula.evaluate.diversity`, `simula.corpus.export`. All other MCP tools are denied by default per MAS MGF pillar 1.

12.14 Implementation tasks for this domain

The tasks below are intent-based: each task states the outcome a future agent must achieve and the location in the existing codebase where the work belongs. None of them contains code; each task binds to the Simula sections that define the required behaviour.

Task S1 — HANA schema extraction.

Intent. Realise the HANA schema extraction pipeline in chapter 3, covering MCP discovery, table enumeration, column extraction, and registry population.

ADR: Why Direct SQL Metadata?

We chose direct SQL-metadata extraction over high-level MCP discovery for the baseline to ensure we capture SAP-specific table types (Global Temporary, Row-Store) that standard MCP tools often omit.

In-scope. Each of the four stages in the pipeline figure (3.1); schema registry writer; per-table dimension/measure classification. **Out-of-scope.** Non-HANA backends. **Paths to extend.** `src/training/pipeline/` extraction module; registry writer that emits entries conforming to the schema-registry record. **Definition of done.** Running extraction against a HANA Cloud catalogue produces a registry populated with every in-scope table; registry entries validate against the schema-registry record.

Autonomous Verification (Agent Logic):

```
// Verify Task S1
{
  "test_fixture": "tests/fixtures/simula/hana_mock.sql",
  "expected_outcome": "table_count == 14",
  "validation_command": "python src/simula/extract.py --verify-registry"
}
```

Verification. Pipeline dry-run against the fixture store described in chapter 9 produces the expected registry entries.

Task S2 — Taxonomy generation (Algorithm 1).

Intent. Implement Algorithm 1 (reasoning-driven taxonomy generation) from chapter 4 in full, including all four steps (factor identification, breadth-first expansion, best-of-N proposal, critic refinement). **In-scope.** Per-step module with the checkpoints in 4.1; taxonomy structure writer. **Out-of-scope.** Manual taxonomy authoring UI. **Paths to extend.** Taxonomy engine colocated with the pipeline. **Definition of done.** Running Algorithm 1 against an extracted registry produces a taxonomy that validates against the taxonomy structure schema and passes the coverage check in chapter 7. **Verification.** Taxonomy engine unit tests pass; the coverage evaluator reports coverage within the chapter's target band.

Task S3 — Agentic data synthesis (Algorithm 2).

Intent. Implement Algorithm 2 (agentic data synthesis) from chapter 5, including all four steps (mix sampling, meta-prompt generation, complexification, example generation) and the double-critic validation. **In-scope.** The agent roster implied by 5.1; vLLM TurboQuant routing for all LLM calls; complexification step and the two critic agents. **Out-of-scope.** Non-agentic synthesis paths. **Paths to extend.** Generation engine in `src/training/pipeline/`; vLLM client wrapper. **Definition of done.** Running Algorithm 2 against a taxonomy produces `TrainingExample` records that pass both critics and validate against the training-example record schema. **Verification.** Generation unit tests pass on fixture inputs; double-critic agreement rate meets the threshold in chapter 7.

Task S4 — Environment and CLI.

Intent. Deliver the environment and CLI surfaces specified in chapter 6, including the configuration table, CLI usage, and output files listed in 6.2. **In-scope.** Configuration loader; CLI

subcommands for each pipeline stage; output-file writers. **Out-of-scope.** IDE integrations. **Paths to extend.** CLI module in `src/training/pipeline/`. **Definition of done.** Each configuration key in the configuration table is read and respected; the CLI runs end-to-end and produces every output file in 6.2. **Verification.** CLI smoke tests pass; output files match the schemas declared for them.

Task S5 — Evaluation framework.

Intent. Deliver the evaluation framework in chapter 7, including calibrated complexity scoring (Algorithm 3, Elo), taxonomic coverage evaluation, embedding-based diversity, and critic-calibration validation. **In-scope.** Each evaluator as a runnable function; report writers. **Out-of-scope.** Real-time evaluation dashboards beyond what the chapter defines. **Paths to extend.** Evaluation module in `src/training/pipeline/`. **Definition of done.** Each evaluator runs against a fixture corpus and emits a report that validates against the corresponding report schema. **Verification.** Evaluator unit tests pass; reports reproduce under fixed seeds.

Task S6 — Test fixtures and testing.

Intent. Deliver the test fixtures specification in chapter 9 and the testing specification in chapter 10 at the coverage thresholds in 10.1. **In-scope.** Invoice, OCR, taxonomy, and training-example fixtures; unit tests for taxonomy traversal and generation; CI wiring. **Out-of-scope.** Performance benchmarking infrastructure. **Paths to extend.** Test directory colocated with the Simula pipeline code. **Definition of done.** Coverage meets 10.1; CI gates block regressions. **Verification.** `make spec-simula` remains green; the Simula test entry points all run.

12.15 Governance obligations applied to this domain

The regulations domain wraps Simula at runtime. Four obligations apply without exception; each is stated as a concrete hook in the Simula codebase.

- **MAS MGF four pillars** (see regulations chapter 3 in `docs/latex/specs/regulations/chapters/03-m` Pillar 1 (assess and bound): every Simula MCP tool is declared in the allow-list in §12.13, deny-by-default. Pillar 2 (design and deploy): the pipeline stages in chapter 5 are traceable end-to-end via the per-request identity attribution described below. Pillar 3 (operate and monitor) and pillar 4 (govern and disclose) are satisfied by emitting the evidence records produced by the Simula evaluation framework (chapter 7) into the shared governance store.
- **Identity attribution** (regulations chapter 10, `10-identity-attribution.tex`). Every MCP call and every pipeline run carries the persona id, agent id, request id, and tenant id. The gateway is the attachment point; Simula code propagates the identity envelope without mutating it.
- **Emergent-capability monitoring** (regulations chapter 8, `08-monitoring-spec.tex`). Extraction, taxonomy generation, synthesis, and evaluation steps are instrumented with the monitoring metrics specified in that chapter; metrics flow to the monitoring sink defined there.
- **Conformance tooling** (regulations chapter 6, `06-conformance-tooling.tex`). The Simula service exposes AI Verify 2.0 plug-in hooks and Moonshot CI/CD 1.1.0 benchmark entry points as required by that chapter.

12.16 Cross-spec integration — full detail

12.16.1 With Arabic AP

Simula consumes Arabic's schemas and fixtures as training input. The contract is schema-in, `TrainingExample`-out: Simula reads `invoice.schema.json`, `vat-checklist.schema.json`, and the Arabic fixture store; it produces `TrainingExample` records. Simula does not call Arabic MCP tools at runtime and does not share state with the Arabic workflow. No cross-domain operational envelope is introduced.

12.16.2 With Trial Balance

Simula consumes TB's schemas and fixtures as training input. The contract is schema-in, `TrainingExample`-out: Simula reads the TB schema set and the TB fixture store; it produces `TrainingExample` records. Simula does not call TB MCP tools at runtime and does not share state with the TB workflow.

12.16.3 With Regulations

Regulations wraps Simula at runtime. Every Simula MCP call is brokered through the regulations governance gate, which applies pillars 1–4 of MAS MGF, attaches the identity envelope, and routes emergent-capability metrics to the monitoring store. The Simula service code does not call regulations directly; the gateway layer enforces this. Simula also consumes regulations' schemas (requirement, conformance-tool, monitoring metrics, identity and audit) as training input.

12.16.4 With Simula

Simula exports the training-example record, the schema-registry record, the taxonomy structure, and the enum vocabularies under `structured/schema/` (chapter 2); registers the MCP tools enumerated in §12.13; and applies the four governance hooks in §12.15 to every call it serves. External consumers see Simula only through these contracts and through the corpus it emits; there are no private back-channels.

12.17 Wave assignment and sequencing

Implementation runs in three waves.

- **Wave 1 — foundations.** Unified schema registry entry; persona-gated surface for the Training Engineer; identity attribution envelope flowing through the gateway; HANA extraction pipeline (Task S1). Prerequisites: the regulations chapter 10 identity schema must be frozen.
- **Wave 2 — Simula core build-out.** Tasks S2, S3, S4, S5. Prerequisites: Wave 1 complete; vLLM TurboQuant endpoint reachable; Arabic, TB, and regulations schema sets published.
- **Wave 3 — hardening and conformance.** Task S6 and full conformance-tooling hook-up. Prerequisites: Wave 2 complete; regulations Wave 2 conformance tooling integrated; monitoring sink live.

Simula occupies Wave 1 for its extraction pipeline (because other domains depend on Simula's fixtures being reachable), Wave 2 for its core build-out, and Wave 3 for its testing and conformance surfaces.

12.18 Acceptance criteria (platform-wide)

When all four chapter 12 instruction sets are implemented, the platform satisfies the following invariants:

1. **One instance.** A single codebase runs a single logical instance; no domain ships as a separate product.
2. **Persona-gated surfaces.** The four personas each reach a tailored UI on the shared shell; no route, MCP tool, or data product is visible outside its persona's entitlement set.
3. **Unified registry.** Every schema in use is listed in the ODPS 4.1 registry under its owning domain; validators resolve `$ref`s through the unified registry only.
4. **No schema merges.** The two `requirement.schema.json` files in TB and regulations remain distinct; no cross-domain composite schemas are produced.
5. **No cross-domain operational envelope.** Runtime orchestration stays within each domain's service; only Simula consumes other domains' outputs, and only as training input.
6. **Governance wraps all three non-regulations domains.** Arabic, TB, and Simula service calls pass through the regulations runtime wrapper without exception.

12.19 References

Within-spec references: chapter 1 (overview), chapter 2 (data schema), chapter 3 (HANA extraction pipeline), chapter 4 (taxonomy engine, Algorithm 1), chapter 5 (generation engine, Algorithm 2), chapter 6 (environment and CLI), chapter 7 (evaluation framework), chapter 9 (test fixtures), chapter 10 (testing specification), chapter 11 (project roadmap).

Peer chapter 12 instruction sets for the other three domains (each is self-contained; consult directly when operating in that domain):

- Arabic AP: `docs/latex/specs/arabic/chapters/12-implementation-instructions.tex`.
- Trial Balance: `docs/latex/specs/tb/chapters/12-implementation-instructions.tex`.
- Regulations: `docs/latex/specs/regulations/chapters/12-implementation-instructions.tex`.

Shared platform artefacts:

- ODPS 4.1 registry: `src/intelligence/ai-core-pal/data_products/registry.yaml`.
- Schema pipeline and resolver: `src/intelligence/schema_pipeline/`.
- Training pipeline (Simula): `src/training/pipeline/`.
- Shared workspace shell: `generativeUI/training-webcomponents-ngx`.

Chapter 13

Global Wave-Dependency Manifest

13.1 Critical Path Visualization (Waves 1–3)

The following chart defines the normative sequencing for implementation agents. A task in a later wave cannot be marked COMPLETE until its prerequisites in an earlier wave are validated.

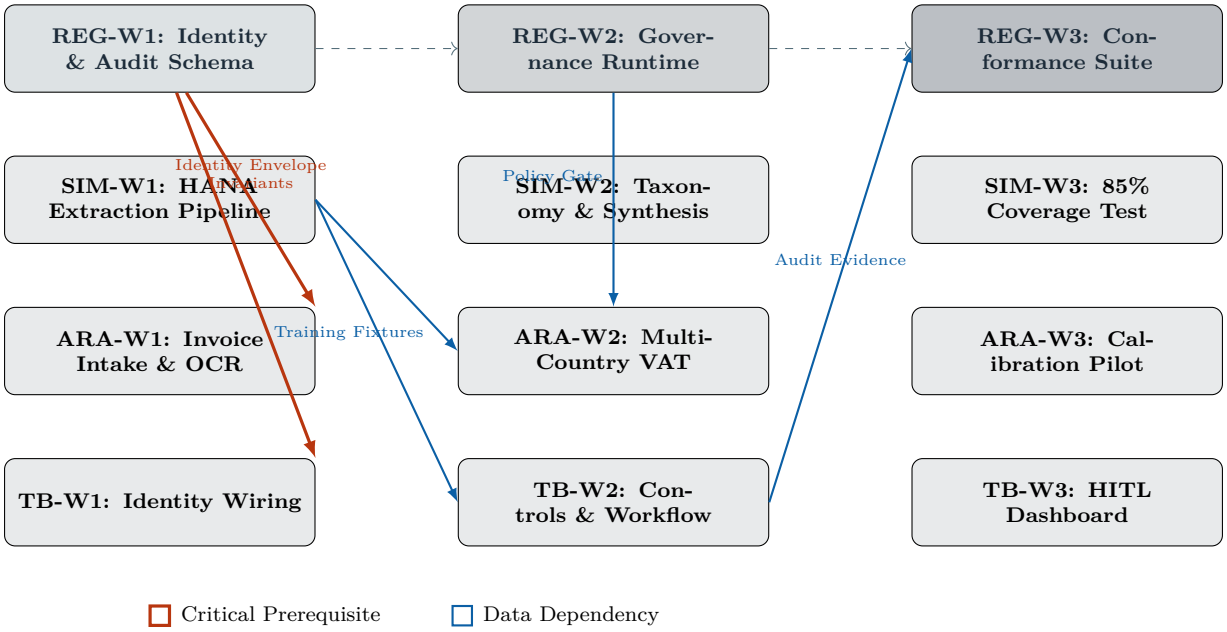

Figure 13.1: Cross-specification dependency graph: The blueprint for platform rollout.

13.2 Inter-Domain Prerequisite Matrix

ADR: Dependency Enforcement

Implementation agents are strictly forbidden from "Fast-Forwarding" into Wave 2 if Wave 1 critical prerequisites (marked in **Gold**) are in a GAP or PARTIAL state.

Table 13.1: Normative Cross-Domain Prerequisites.

Target Task	Prerequisite	Ref ID	Rationale
ARA-W1	REG-W1 (Identity)	`REG-IDENTITY-01`	Invoice intake must be audit-traced from Day 1.
TB-W2	SIM-W1 (Extraction)	`SIM-EXTRACT-01`	Anomaly detection requires HANA schema metadata.
REG-W2	SIM-W1 (Registry)	`SIM-REGISTRY-01`	Governance monitoring relies on the schema registry.
ARA-W2	REG-W2 (Pillar 1)	`REG-MGF-P1`	VAT release is an HITL gate governed by MAS Pillar 1.
TB-W3	ARA-W2 (Posted Invoices)	`ARA-AP-POST`	Review dashboard depends on posted AP data from Arabic domain.